Infrastructure & Operations

Polycarpic — a virtual Single Family Office

Owner: `devops-engineer` agent **Status:** Approved **Version:** 1.3.10 **Last updated:** 2026-11-13

CONTENTS

1. Targets & Topology
2. Resource Allocation
3. Identity, Keys & TLS
4. Environments & CI/CD
5. Update & Patch Schedule
6. Disaster Recovery & Monitoring
7. Deployment Runbook
8. Operations Runbook
9. Rulings & Asks on Other Owners
10. Open Questions

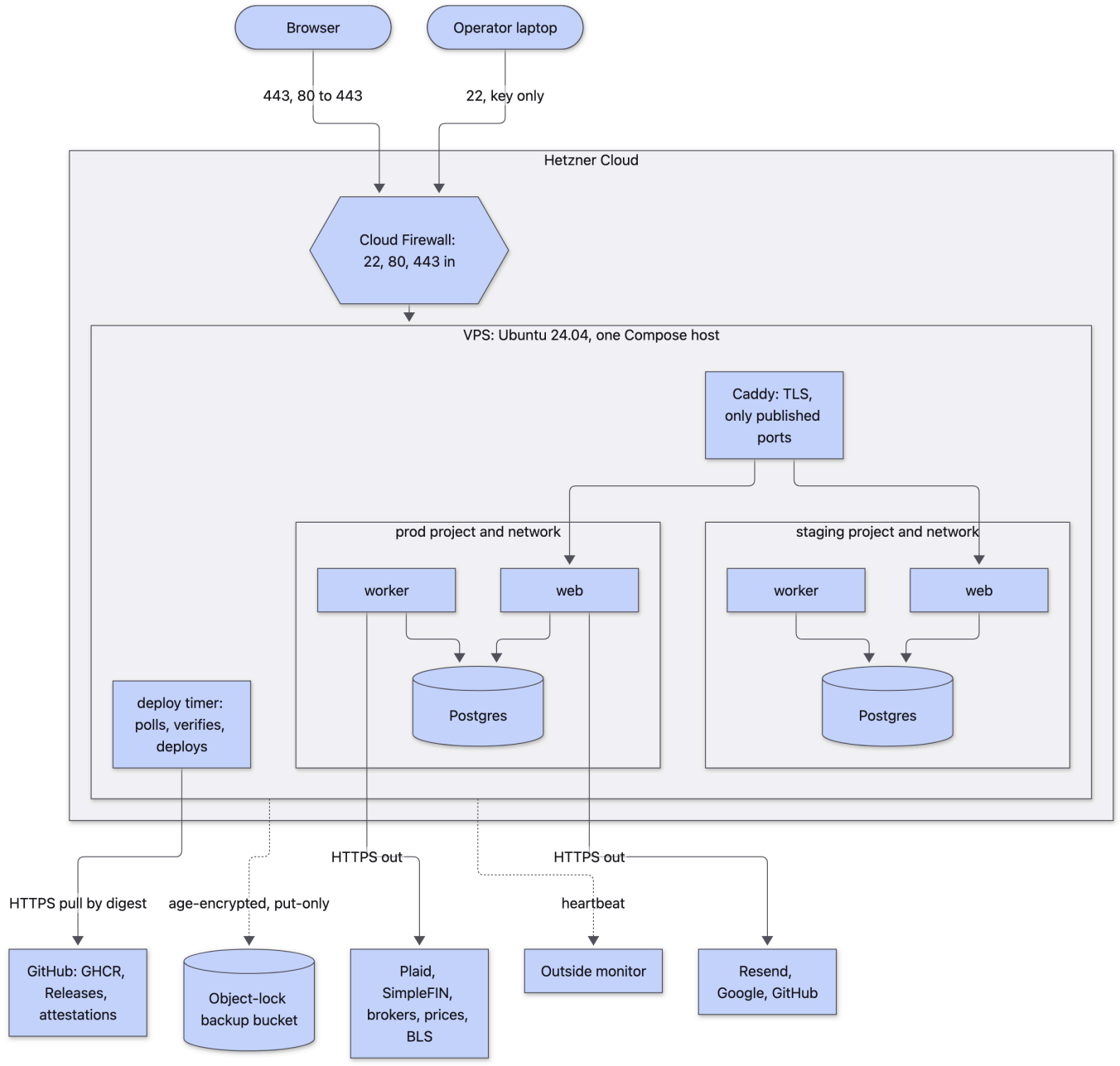
1. Targets & Topology

Order: a Hetzner VPS, then a local-network box, then Vercel + Neon. ARCH §7 says what each is; SECURITY C-70 to C-77 say why it is built this way; this document says how to stand it up and run it. The three are deployment targets of one application, not three architectures.

1.1 Reading this document as the operator

- You are one person running one instance for your own household (PRD §3). Nothing here is written for hosting other people, and nothing here makes that safe (SECURITY R-6).
- If you only read two sections, read §7 (to stand it up) and §8 (to keep it alive). Everything else is the reasoning behind them.
- Terms: the **box** is the machine running the containers; the **operator** is you; a **digest** is the exact fingerprint of one build of the app image (`sha256:…`); **compose** is the tool that starts the containers listed in `compose.yml` .

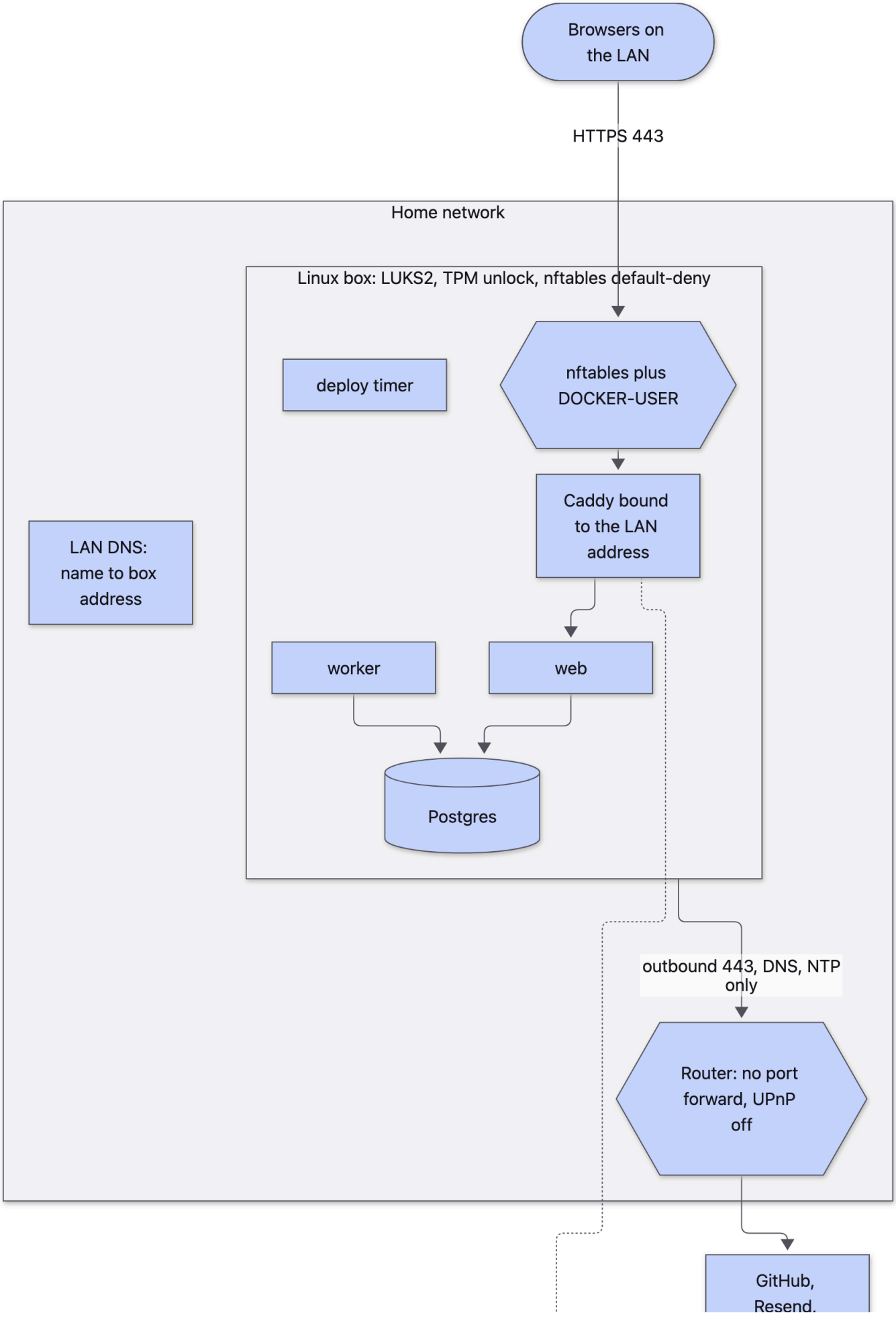
1.2 Primary: Hetzner VPS (Docker Compose)

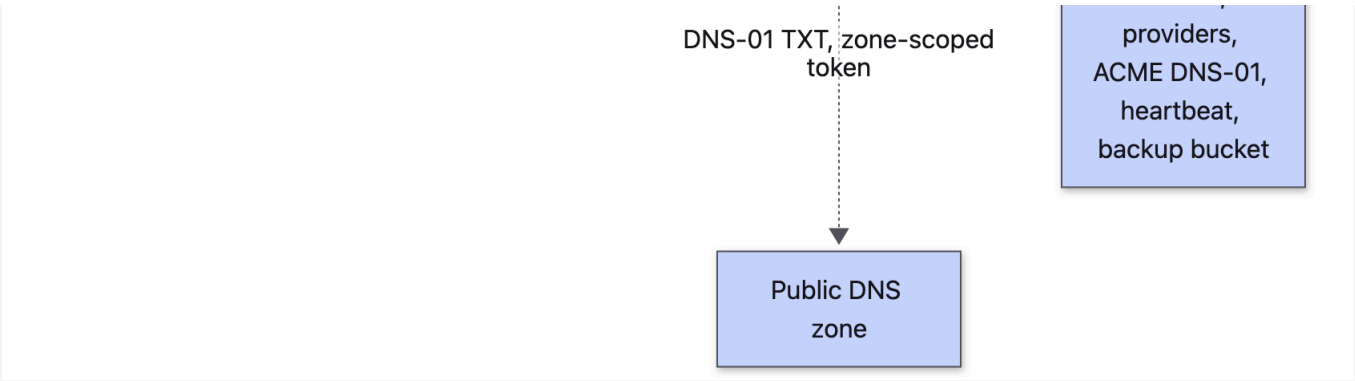


Concern	Decision
Machine	One Hetzner Cloud server, 4 vCPU / 8 GB / shared x86, Ubuntu Server 24.04 LTS (§2.1). Location: **Falkenstein (`fsn1`)**, EU. Reason: it is the default Hetzner location with the widest plan availability, and the latency difference to Nuremberg or Helsinki is invisible for this app; Helsinki has no concrete advantage here. EU prices and included traffic are cheaper than the US locations (figures unmeasured; read them on the order page). IPv4 and IPv6 both enabled on the server; both are covered by the firewall rules.

Concern	Decision
Ports	The Cloud Firewall (outside the host, C-71) admits 22, 80 and 443 only. On the host only Caddy publishes ports; the compose-policy lint (C-70) and the post-deploy outside-in scan (§7.1 step 15) both enforce this.
SSH	Key-only, no root login, no password auth. Port 22 stays open to the world because a home connection's address changes. The key and the unattended-upgrade posture are the control (§3.1). Restricting 22 to your current address in the firewall is an option, not the default, and a mistake there locks you out (§8.9).
DNS	Two `A` / `AAAA` records: `app.<domain>` to prod, `staging.<domain>` to staging, both pointing at the one server. Public DNS is Cloudflare's free DNS in DNS-only mode (grey cloud, no proxy), with the registrar unchanged.
TLS	Caddy obtains and renews public certificates by ACME HTTP-01 (port 80). No operator action. Staging sits behind Caddy basic auth plus an IP allowlist (ARCH §7.1).
Third parties on this path	GitHub (image registry, release metadata, attestations; pulled by the box, §4.4); Resend (outbound mail); Plaid, SimpleFIN, direct broker APIs, the EOD price provider (OQ-5) and BLS (outbound, worker); Google and GitHub OAuth (web v1; Sign in with Apple is native-only, POLY-122); the backup bucket; one outside monitor. Each is outbound-only from the box except the browser's own redirect.

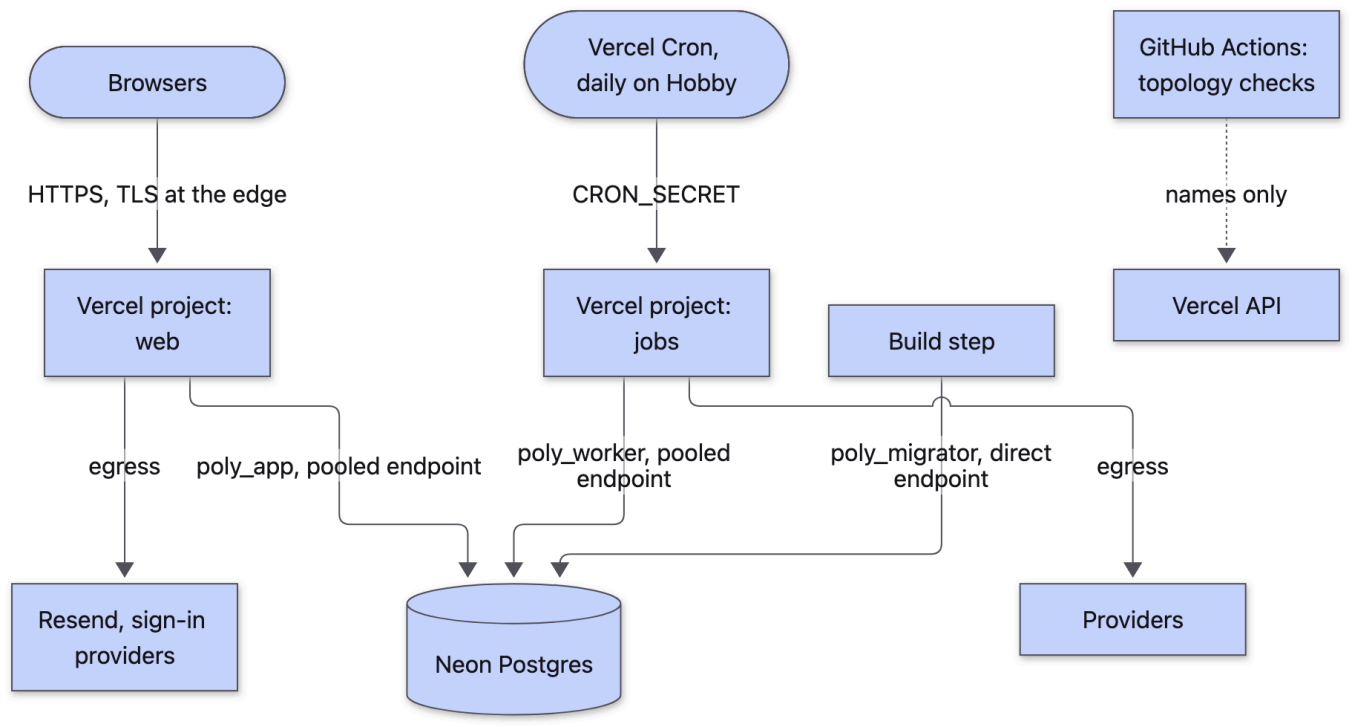
1.3 Secondary: local-network box (outbound-only)


Browsers on the LAN
HTTPS 443
Home network
Linux box: LUKS2, TPM unlock, nftables default-deny
deploy timer
nftables plus DOCKER-USER
Caddy bound to the LAN address
LAN DNS: name to box address
worker
web
Postgres
outbound 443, DNS, NTP only
Router: no port forward, UPnP off
GitHub, Resend,



Concern	Decision
Who may reach it	Browsers on the LAN only (C-76). No inbound path from the internet exists: the router forwards nothing, UPnP and NAT-PMP are off, the host drops inbound IPv6, and nftables allows 443 (80 redirects) and SSH from the LAN CIDR only. Away from home there is no access in v1.
Outbound allowlist	DNS, NTP and TCP 443 only (C-76). **Consequence found here:** an SFTP upload to a Hetzner Storage Box uses port 22 or 23 and is blocked, which is one reason §9 rules the backup destination to an S3-compatible bucket over 443 on every target.
DNS	Real-domain mode (default): the name is `polycarpic.home.<domain>`, defined only on the LAN (router DNS override, or one hosts-file line per device). There is no public `A` or `AAAA` record for it. The public zone holds only short-lived `_acme-challenge` TXT records written by Caddy. `.local` mode: the name is `polycarpic.local`, answered by mDNS or the same LAN DNS override.
TLS	Two modes selected by `TLS_MODE` (§3.4): `real-domain` (DNS-01 wildcard from a public ACME issuer) or `private-ca` (name-constrained operator CA). The app refuses to start on a mismatch (ARCH §7.8, SECURITY C-76).
Third parties	Same as §1.2 minus nothing; all are outbound over 443. OAuth sign-in works in real-domain mode and is disabled server-side in private-CA mode. A verification email link only opens from the LAN.
Hardware	Any x86-64 mini PC or small server with a TPM 2.0, UEFI Secure Boot, 8 GB RAM or more, a 256 GB or larger SSD, and wired Ethernet. Put it on a small UPS. Buying advice is in §7.2 step 1.

1.4 Tertiary: Vercel + Neon (serverless fallback)



Concern	Decision
Shape	Two Vercel projects from one repo, one Neon project (ARCH §7.6, SECURITY C-61). No Caddy, no worker container, no deploy timer: Vercel deploys on a Git push to the production branch/tag, and migrations run as a build step.
Connections	Runtime uses Neon's **pooled** endpoint (PgBouncer, transaction mode). The migrate build step uses the **direct** endpoint, because a migration that sets role or takes advisory locks must not run through a transaction-mode pooler (§9.1 ruling D-6).
Freshness	Hobby cron is once a day. Imports and period-end jobs run daily at best (ARCH §7.6, accepted).
Backups	OQ-26 answered in §6.6: Neon's restore window plus an operator-run weekly encrypted export; no nightly automated export and no credential in GitHub.
When to use it	When you have no machine you trust to keep running, or as a temporary landing place while a primary is rebuilt. It is the weakest target for secrets (SECURITY R-3). Do not link real aggregators here until POLY-77 and the R-3 review.

2. Resource Allocation

*Compose layout, sizing, volumes and storage. Sizing numbers are starting values, **unmeasured** until POLY-M1 produces real metrics (ARCH §7.3); every number is one line in `compose.yml` or the provisioning file.*

2.1 Host

Item	Hetzner VPS	Local box
OS	Ubuntu Server 24.04 LTS	Ubuntu Server 24.04 LTS (same, so one set of instructions)
Size	4 vCPU, 8 GB RAM, the cost-optimised shared-x86 class (ARCH §7.3 names CX32; Hetzner renames its plans, so pick the 4 vCPU / 8 GB shared-x86 plan on the order page and record its name in the ledger). Downgrade to 2 vCPU / 4 GB is a one-line change if metrics allow.	8 GB RAM minimum, 16 GB comfortable
Root disk	Plan default (about 80 GB)	256 GB SSD or larger; root and data are logical volumes inside one TPM-unlocked LUKS2 container (§3.5)
Data volume	A separate Hetzner Cloud Volume of 20 GB, mounted at `/srv/polycarpic`, formatted LUKS2 (§3.2)	A logical volume in the same LUKS2 container as root, mounted at `/srv/polycarpic`, TPM-unlocked (§3.5, §7.2)
Docker	Docker Engine and the Compose plugin from Docker's apt repository, `live-restore` on, `json-file` log driver with `max-size 10m` and `max-file 5`. Upgraded by hand in the monthly window (§5).	

2.2 Compose layout

Two compose projects on the Hetzner host, `polycarpic-prod` and `polycarpic-staging`, each with its own network, plus one shared `edge` project holding Caddy. The local box runs only `polycarpic-prod` and `edge`. Caddy joins each project's network through an external network named per project, so a staging container has no route to a prod container (TB-10, C-70). Two

more networks exist only in the prod project (table below): one that carries the backup login and no internet, one that carries the internet and no database.

Service	Project	Image	Entrypoint	Memory cap (start)	Secrets mounted
`caddy`	edge	Custom Caddy build (DNS plugin on the local box), pinned by digest, built and scanned in CI	default	128 MB	Home real-domain mode: `dns_api_token`. Staging: `staging_basic_auth`. Nothing else.
`postgres`	prod	Official `postgres:17.10` image (major 17 from `infra/postgres-major`, minor 17.10 as measured in the POLY-76 spike; §6.1a) plus the `age` binary (a two-line derivative whose `FROM` line is `postgres:17.10@sha256:…`, built, scanned and attested in CI so `archive_command` can encrypt, §6.1), by digest	default	3 GB, `shared_buffers` 768 MB	`db_superuser_password`. The config fragment, `pg_hba.conf` and the age *public* key are mounted as config files. Not `db_password_backup`: `provision.sh` sets that role's password through the superuser socket.
`web`	prod	The app image, by digest	`web`	768 MB	`db_password_app`, `auth_secret`, OAuth client secrets, `resend_api_key`, `plaid_secret` (token exchange). No keyring.

Service	Project	Image	Entrypoint	Memory cap (start)	Secrets mounted
`worker`	prod	The same app image	`worker`	768 MB	`db_password_worker`, `credential_keyring`, provider and price/BLS keys, `resend_api_key`
`migrate`	prod	The same app image	`migrate`, one-off	512 MB	`db_password_migrator` only
`operator`	prod	The same app image	`polycarpic cli`, one-shot (`restart: "no"`, profile `operator`, `logging: {driver: "none"}`, started only by `docker compose --profile operator run --rm operator …`)	256 MB	`db_password_operator` only. No long-running service mounts it; the compose-policy lint's `operator-one-shot` rule refuses it anywhere else and requires the one-shot shape and the `none` log driver, because the CLI's stdout carries the invitation link and must not reach a log file. It connects as `poly_operator`, which holds EXECUTE on `app.issue_operator_invitation` and nothing else.
`postgres`, `web`, `worker`,	staging	Same images	same	1 GB / 384 MB / 384 MB / 512	Distinct values from `/etc/polycarpic/staging-secrets/`, its own keyring, Plaid sandbox secret only (C-77)

Service	Project	Image	Entrypoint	Memory cap (start)	Secrets mounted
`migrate`, `operator`				MB / 256 MB	
`backup`	prod	The same app image, or a small tools image (`age`, `pg_basebackup`, `pg_dump`), by digest. **No `rclone`**, no bucket client.	`backup`, run by a systemd timer; also run once by the deploy script for the pre-migration backup (§4.5)	256 MB	`db_password_backup` and the age *public* key as config. Nothing else; never the private key, never the bucket credential. Networks: `backup-net` only (internal, no internet).
`uploader`	prod	A small tools image (`rclone` and a shell), by digest	`uploader`, run by a systemd timer every minute and after each backup run	128 MB	`backup_store_credential` (the put-only pair) and `heartbeat_url`. **No database credential**, no route to Postgres. Network: `upload-net` only (internet allowed; on the home box the host allowlist still limits it to 443).

Networks (prod project). Subnets are fixed in the compose file because `pg_hba.conf` matches a subnet, and a container address is not stable. The Docker daemon is told (`default-address-pools` in `daemon.json`, base `10.200.0.0/16`) to allocate everything else elsewhere, so the fixed ranges never collide with an auto-assigned one (unmeasured; checked by `provision-check`).

Network	Subnet (start)	Members	Properties
`polycarpic-prod_default`	`10.201.1.0/24`	`postgres`, `web`, `worker`, `migrate`, `operator`	The application network. `backup`, `uploader` and `caddy` are **not** members. Staging's default network is `polycarpic-staging_default`, fixed at `10.202.1.0/24`; staging has no `backup-net` and no `upload-net`.
`polycarpic-prod-edge`, `polycarpic-staging-edge`	`10.201.30.0/28`, `10.202.30.0/28`	`caddy` and that project's `web` (network alias `prod-web` or `staging-web`), nothing else	Defined in `edge.yml`, `external` in the project files. Outside every `pg_hba.conf` allow range and the `10.200.0.0/16` pool. Caddy is on no `_default` network, so it has no route to Postgres.
`backup-net`	`10.201.10.0/29`	`postgres`, `backup` and nothing else	`internal: true`: no route out. Fixed subnet. The only place `poly_backup` may connect from (C-79).
`upload-net`	`10.201.20.0/29`	`uploader` only	Ordinary outbound network. Not shared with Postgres, so the container that holds the bucket key cannot reach the database even if it somehow held a credential.

`pg_hba.conf` (mounted, not edited in place). The four C-79 lines come first so they match first; the rest follow. The login lines for the three app roles name the default project subnet, never `all`.

```
host  replication  poly_backup    10.201.10.0/29  scram-sha-256
host  polycarpic   poly_backup    10.201.10.0/29  scram-sha-256
host  all          poly_backup    all             reject
host  replication  all            all             reject
host  polycarpic   poly_app       10.201.1.0/24   scram-sha-256
host  polycarpic   poly_worker    10.201.1.0/24   scram-sha-256
host  polycarpic   poly_migrator  10.201.1.0/24   scram-sha-256
host  polycarpic   poly_operator  10.201.1.0/24   scram-sha-256
```

```
host  all          all            all             reject
local all          postgres                       peer
```

How the two halves hand over. `backup` pipes each artifact through `age` as it is taken, so plaintext never reaches disk, and writes only `.age` files into `spool/dump/` or `spool/premigration/` (bind mounts, `backup` alone writes there). Postgres''s `archive_command` does the same for WAL into `spool/wal/`. The `uploader` mounts the whole spool and a state directory, sends each file, verifies it, deletes it from the spool and records the result (§6.1, §6.5). Neither can do the other's job: `backup` has no route out and no bucket key; `uploader` has no database credential and no route to Postgres. Nothing outside these two containers reads or writes the spool except root on the host.

Planned total caps are about 7.4 GB (adding the 128 MB `uploader`) for a prod+staging host that is never fully busy at once; the host keeps the remainder. Unmeasured. If `docker stats` after POLY-M1 shows the caps are hit, raise the plan size before raising a cap.

- Only `caddy` has a `ports:` mapping. Caddy listens on 8080 and 8443 inside the container (no `NET_BIND_SERVICE` under `cap_drop: [ALL]`); `ports:` maps `${POLY_BIND_ADDR:?}:80:8080` and `${POLY_BIND_ADDR:?}:443:8443`. `POLY_BIND_ADDR` has no default, so a missing value fails the render instead of binding `0.0.0.0`; on the local box it is the LAN address.
- Caddy reads three operator values from its environment, all required: `POLY_DOMAIN` (sites `app.<domain>` and `staging.<domain>`), `POLY_STAGING_ALLOW` (the staging `remote_ip` allowlist) and `POLY_TLS` (the ACME account email on a real host; `internal` only in tests). A missing value fails the render rather than starting Caddy with an unparsable Caddyfile.
- `backup-net`'s gateway, the host, lies inside the `/29` that `pg_hba.conf` admits for `poly_backup`. Host root is already trusted, so this is accepted.
- The one-off services (`migrate`, `backup`, `uploader`) sit behind compose profiles and use `restart: "no"`; every long-running service uses `unless-stopped`.
- Every container: non-root user, `read_only: true` with named `tmpfs` for writable paths, `cap_drop: [ALL]`, `security_opt: [no-new-privileges:true]`, `restart: unless-stopped`, a `healthcheck`. No Docker socket is mounted anywhere.
- The compose-policy lint (C-70) is a CI job that parses the rendered config (`docker compose config`) and fails on any violation of the list above, any secret in `environment:`, any `ports:` outside `caddy`, or any service on both project networks. C-79 adds: only `backup` and `postgres` are on `backup-net`; `backup-net` is `internal: true` with a fixed subnet equal to the one in `pg_hba.conf`; only `backup` mounts `db_password_backup`; only `uploader` mounts `backup_store_credential`; `uploader` is on no network that `postgres` is on.

- Postgres runs with the C-75 settings from a mounted `postgresql.conf` fragment: `log_parameter_max_length = 0`, `log_parameter_max_length_on_error = 0`, `log_statement = 'ddl'`, plus `archive_mode = on`, `archive_timeout = 300` and an `archive_command` that encrypts each segment with `age` straight into `spool/wal/` (§6.1). C-79 adds: `max_replication_slots = 2`, `max_wal_senders = 3`, `max_slot_wal_keep_size = 2GB` (unmeasured, tuned with OQ-INFRA-5) so a slot can never fill the disk, `log_connections = on`, and `password_encryption = scram-sha-256`. The `image` job starts the real `postgres` service of `prod.yml` (same mounts and command, no `-c` flags) and runs `SHOW` on each, checks `pg_settings.sourcefile` is the mounted fragment, and checks `ALTER SYSTEM` is refused; `provision-check` runs `SHOW` on the deployed server.

2.3 Volumes, storage and buckets

Where	What	Notes
`/srv/polycarpic/pg-prod`, `/srv/polycarpic/pg-staging`	Postgres data directories	On the LUKS volume (C-72).
`/srv/polycarpic/spool`	Encrypted-but-not-yet-uploaded backup files and WAL, in `dump/`, `wal/` and `premigration/`	Same LUKS volume. **Holds only `.age` files** (and a `.sha256` sibling once uploaded); `provision-check` fails on anything else. Pruned by the `uploader` after a verified upload; pre-migration files are kept 7 days. A spool older than 2 hours alerts (§6.5).
`/srv/polycarpic/caddy`	Caddy's certificate and ACME account storage	Losing it only forces reissue, but keep it to stay inside Let's Encrypt rate limits.
`/var/lib/polycarpic/state/`	Host state files: `last-upload` (written by the `uploader` only after it has verified the object), `last-dump`, `jobs.log`	root:root 0755, no secrets, a few lines each. Read by `polycarpic-ops status`, the alerting and the heartbeat script (§6.5). Mounted read-write into `backup` (its two files) and `uploader` (its one), nowhere else. On the home box it sits on the encrypted root.

Where	What	Notes
`/etc/polycarpic/secrets/`, `/etc/polycarpic/staging-secrets/`	Secret files (§3.1)	root-owned. One subdirectory per service (`<dir>/<service>/<name>`), each `root:<that service's gid>` mode 0750 with its files 0440; the top directory is root 0700. Not on the data volume and never in a backup.
`/opt/polycarpic/`	The deploy script, the timer unit, per-release ops bundles (`releases/<digest>/`), the `desired-digest` and `current-digest` files, `deploy.log`	No secrets. The ops bundle (compose files, Caddyfile, scripts) is extracted from the verified image, so the same attestation covers it (§4.4).
Backup bucket	A Backblaze B2 bucket with Object Lock in compliance mode, default retention 35 days, in B2's EU region (§6.1, D-3)	One bucket per instance, reached through B2's S3-compatible endpoint with `rclone`. Cost: pay-as-you-go storage (a few USD per TB-month, a small free allowance) so cents per month at household volume; unmeasured, confirm on B2's pricing page at signup.
Off-box copies the operator holds	The age private key, the keyring offline copy, the LUKS recovery passphrase, the private CA root (if used), registrar and DNS logins	In the operator's password manager plus a printed recovery sheet in a safe place (§3.6).

3. Identity, Keys & TLS

Who holds what, and what each holder can do with it. Implements SECURITY §6 (secrets), C-71/C-72/C-74/C-76. Compliance boundary: one operator, one household, no hosting for others (PRD §3, SECURITY §1, R-6).

3.1 Who holds what

Holder	Holds	Can do	Cannot do
Operator (you)	SSH private key (one per laptop, passphrase-protected, hardware-backed if you own a security key); password manager vault; age *private* key; keyring offline copy; LUKS recovery passphrase; registrar, DNS, GitHub, Hetzner, bucket and Resend logins	Everything. You are the root of trust (SECURITY R-1).	—
The box (root)	Secret files; age *public* key; bucket *put-only* credential; read-only GitHub token (or none if the package is public)	Run the app; encrypt and upload backups; pull verified releases	Decrypt any backup; delete or overwrite any stored backup; deploy anything unverified
`web` container	Its own secret files, public sealing keys	Seal credentials	Open credentials; read the keyring or the backup key
`worker` container	The keyring and provider secrets	Open credentials, inside one call frame	Reach the internet beyond what the host firewall allows
`backup` container	`db_password_backup`; the age public key	Read the whole cluster as `poly_backup`, from `backup-net` only; write encrypted files to the spool	Reach the internet; hold or use the bucket key; decrypt anything

Holder	Holds	Can do	Cannot do
`uploader` container	The put-only bucket pair; the heartbeat URL	Send spool files to the bucket and ping the monitor	Reach Postgres; hold any database credential; read plaintext (the spool has none)
Caddy container	Only the DNS-01 token (home) or the staging basic-auth hash	Terminate TLS	Read any app secret
GitHub Actions	Build and attest permissions; no credential that can reach any box	Publish an attested image	Log in to, or push to, a box (C-83)
Backup bucket credentials	Two, separate: **put-only** on the box; **read and list** on the operator's drill machine only	—	Neither can delete or shorten retention; Object Lock enforces that at the store

3.2 Secret files on the host

Every operator secret is a file under `/etc/polycarpic/secrets/<service>/` (staging: `staging-secrets/<service>/`, different values, no shared file). Plain Compose bind-mounts `file:` secrets and ignores `uid`, `gid` and `mode`, so access is decided by host ownership: every service runs as its own numeric uid and its own gid (no `group_add`), its directory is `root:<gid>` 0750 and its files 0440, so no service can read another's directory and nothing is world-readable. A secret two services read (`resend_api_key`) is two files, one in each service's directory. Compose mounts each as a top-level `<name>__<service>` secret at `/run/secrets/<name>` into only the container listed in §2.2; the compose-policy lint enforces the layout and the matrix. The app reads them through the `*_FILE` convention and refuses to start if one is missing or if a secret-looking environment variable is set in production (SECURITY §6.2).

File	Contents	Created by
`db_superuser_password`, `db_password_app`, `db_password_worker`, `db_password_migrator`, `db_password_operator`, `db_password_backup`	Random, 32 bytes base64 each	`provision.sh` generates them on the box and sets each role's password in Postgres; you never type them. `db_password_backup` is mounted into `backup` alone.
`auth_secret`	BetterAuth secret, random 32 bytes. Mounted into `web` alone. Versioned (§8.1).	`provision.sh`
`credential_keyring`	X25519 private keys by `key_id`	`polycarpic secrets init`, run once in the worker; you then save the offline copy (§7.1 step 14)
`oauth_google_secret`, `oauth_github_secret`, `resend_api_key`, `plaid_secret`, broker client secrets, `price_api_key`, `bls_api_key`	Provider-issued	You paste each with `polycarpic-ops set-secret <name>`, which reads from the terminal without echo and writes 0440 into each consuming service's directory
`backup_store_credential`	The put-only access key id and secret for the bucket. Mounted into `uploader` alone.	You, same command
`heartbeat_url`	The Healthchecks.io ping base URL with the project's ping key (anyone holding it can fake a heartbeat)	You, same command
`dns_api_token` (home, real-domain mode)	DNS-edit token scoped to one zone	You (§3.4)

File	Contents	Created by
`staging_basic_auth`	A bcrypt hash, not the password	`polycarpic-ops staging-auth`

Backups never contain this directory. The Hetzner data volume's LUKS key is a file on the root disk (`/etc/polycarpic/luks.key`, 0400). That protects against a detached or snapshotted volume only; it does not protect against someone who gets the running server or its root disk. Said plainly so nobody believes more than that. The home box does better (TPM, §3.5).

3.3 SSH, GitHub token, signing

- **Operator SSH.** One `ed25519` key per laptop, listed in `~/.ssh/authorized_keys` of a non-root sudo user `polyops`. Root login and password auth are off. Adding or removing a laptop is §8.8. Hetzner's console is the break-glass path if you lose every key.
- **The box's GitHub credential (ARCH §8.1).** A fine-grained personal access token, this repository only, `contents:read` and `packages:read`, expiry 90 days, stored as `github_read_token` and used only by the deploy script. If the package and repository become public, delete it. Renewal is §8.4. A reminder is set 14 days before expiry by the same outside monitor (§6.5).
- **Release signing.** The prod tag is accepted only if its SSH signature verifies against `/etc/polycarpic/allowed_signers`. That file is **installed at bootstrap from your own machine, not fetched from the repository**: a copy pulled from the repo would be forgeable by whoever can push to it (D-5).
- **Image trust.** `gh attestation verify` pins the repository and the signer workflow `.github/workflows/build.yml` (§4.4).

3.4 TLS

Target / mode	How the certificate is made	Renewal	What the operator does
Hetzner	Caddy, ACME HTTP-01 on port 80, public issuer (Let's Encrypt by default)	Automatic, about 30 days	Nothing. The health check alerts if the served

Target / mode	How the certificate is made	Renewal	What the operator does
		before expiry	certificate has under 14 days left.
Home, `real-domain`	Caddy with the DNS provider's plugin, ACME DNS-01 for `*.home.<domain>`. Wildcard, so the box's own name never appears in public certificate-transparency logs. The token is scoped to the one zone (Edit DNS on that zone, nothing else) and mounted into Caddy only (R-18).	Automatic	Rotate the token at the DNS provider yearly or on suspicion (§8.3). Optional: watch the zone's CT-log feed.
Home, `private-ca`	An operator CA. **Root**: generated on your laptop, X.509 name-constrained to `polycarpic.local`, key kept offline in the password manager, validity 10 years, never on the box. **Intermediate**: signed by the root, validity 2 years, on the box, issues leaves. Caddy's internal issuer uses the intermediate; leaf validity 7 days, renewed automatically.	Leaves automatic. Intermediate by hand before it expires (§8.5).	Install the root on every device (§7.3). Remove it from every device on decommission or a move to a real domain.
Vercel	Vercel terminates TLS	Automatic	Nothing

Gate item (SECURITY R-20): before the first home-box deploy in private-CA mode, run the name-constraint test of §7.3 step 6 on each browser and OS you use. If any browser ignores the constraint, that browser must not have the root installed, and the box moves to real-domain mode.

Startup mode check: the app derives the required mode from its public origin and refuses to start on a mismatch (suffix test offline, then an SOA query for the zone; SECURITY C-76, ARCH §7.8). INFRA adds only that the deploy script runs `polycarpic check-config` in the `migrate` container before any migration, so a wrong `TLS_MODE` fails the deploy cleanly instead of crash-looping `web`.

3.5 Home-box disk unlock (TPM)

The home box has **one LUKS2 container** holding an LVM volume group; `/` and `/srv/polycarpic` are logical volumes in it (SECURITY C-72, T-85). One container means one unlock, one TPM binding and one recovery passphrase, so the box still restarts unattended. The container has two key slots: a TPM2 slot sealed to the Secure Boot state (PCR 7), and a long recovery passphrase kept offline. **Use PCR 7 only** by default. Adding PCR 11 (the kernel image) makes every kernel update change the measurement and leave the box locked after the automatic reboot, unless the distribution ships signed PCR policies; whether Ubuntu 24.04 does is unmeasured (OQ-INFRA-2).

- **Method (unmeasured, OQ-INFRA-10).** Ubuntu's default initramfs cannot unlock from a TPM by itself. Two candidates, tried in this order at the first home-box deploy: (A) the installer's encrypted LVM layout plus `clevis-luks` with the `tpm2` pin on PCR 7 and `clevis-initramfs`; (B) `dracut` in place of `initramfs-tools`, with `systemd-cryptenroll --tpm2-pcrs=7`. The first method that passes every check in §7.2 steps 6 and 7 is recorded here. If neither passes, stop and return to SecEng and the Principal; no fallback is chosen here.
- **What stays unencrypted:** the EFI partition and `/boot` (kernel and initramfs). Someone with an hour and the box can rebuild an unsigned initramfs there; that residual is accepted (T-85) unless OQ-INFRA-2 finds a way to measure it.
- **Consequence you must know:** with root encrypted the box has no network until it is unlocked, so a failed unlock waits at a console prompt and SSH cannot reach it. Recovery is a keyboard, a monitor and the recovery passphrase (§8.9). Until the unattended-reboot checks of §7.2 step 6 pass, automatic reboots stay off on this box and you reboot it yourself in the monthly window (§8.2).

Firmware checklist, console hardening and verification are in §7.2.

3.6 The recovery sheet

One printed page, kept somewhere that survives a house fire, listing: where the password manager's emergency kit is; the age private key (as the `AGE-SECRET-KEY-…` line); the credential keyring offline copy; the LUKS recovery passphrase; the private CA root key (private-CA mode). **The age key and the keyring are written on separate lines of separate sheets and never in the same file**; a lost keyring costs a relink (RB-4), a lost age key costs every backup (§6.1). Nothing on the sheet is stored on the box or in a backup.

4. Environments & CI/CD

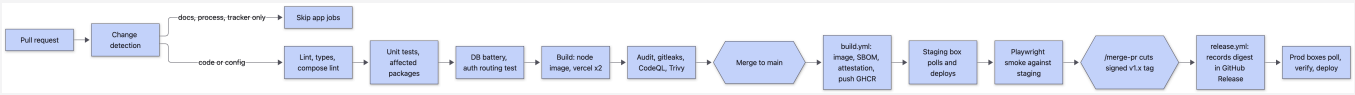
ARCH §7.1 and §8 give the shape; SECURITY §8 (C-80 to C-83) the controls. Budgets: inner loop under 5 minutes, full pipeline under 15. Docs and tracker-only changes never start the app jobs.

4.1 Environments

Environment	Where	Data and secrets	Deploys when
dev	Laptop, local compose (Postgres, web, worker)	`fake` provider. `.env.example` is the only env file in git; names and comments, no values.	By hand
CI	GitHub-hosted runner, fresh Postgres service per run, the same `postgres:17.10` image as prod (§6.1a)	Throwaway secrets generated per run. No repository secret reaches a fork PR (C-81).	Every pull request
staging	The Hetzner VPS, project `polycarpic-staging`, its own database	Fake source plus Plaid Sandbox and the SimpleFIN demo token. Own keyring and own secret directory (C-77). Never real books.	Automatically, every merge to `main`, within about 10 minutes
prod (Hetzner)	Same VPS, project `polycarpic-prod`	Real data from v1.0	A signed `v1.*` tag, within 5 minutes of the release being published. A `0.x` tag is staging-only and prod ignores it.
prod (home box)	Local box, project `polycarpic-prod`	Real data	Same tag rule; the box polls outbound. No staging on the box: it runs releases that staging already proved.
prod (Ver	Two Vercel projects + Neon	Vercel env vars (R-3)	The operator's `infra/vercel/deploy.sh`

Environment	Where	Data and secrets	Deploys when
cel)			on a verified tag (D-11); migration runs in the build step

4.2 Pipeline



4.3 Which jobs run on which paths

Workflows follow the pattern already proven in `.github/workflows/ci.yml`: **no workflow-level `paths:` filter** on anything that is a required check (a required check whose workflow never starts blocks the merge forever); the filter is a `changes` step inside the job and every later step is gated on it, failing closed. Inside the app, Turborepo's affected graph narrows the work to touched packages.

Changed paths	Jobs that run
`docs/**`, `process/**` except the two the tooling tests read, `*.md`	Nothing in the app workflow. Doc lint only.
`packages/*`, `apps/*`	Lint and types; Vitest on affected packages and their dependents; build
`packages/db/**`, any migration, any policy	The above, plus the DB battery on a fresh Postgres at the pinned major: apply every migration as `poly_migrator`, pgTAP RLS and trigger battery, the C-75 config assertion, the pooler-reuse assertion (§9.1 D-6)

Changed paths	Jobs that run
`packages/auth/**`, `packages/tenancy/**`, `better-auth` or `@better-auth/passkey` version	The BetterAuth routing-pool test (ARCH §8, SECURITY A-31, A-33) and the route-policy test (A-34): every one of the routes the pinned versions expose is in exactly one class, a path in no class answers 404, a blocked path answers 404 over HTTP while the same call made server-side still works, and the `drizzleAdapter` lint finds nothing. A dependency bot's bump of either package cannot merge without them.
`compose*.yml`, `Caddyfile`, `infra/**`, `Dockerfile*`, `tests/infra/**`	Compose-policy lint and its fixture suite in the `lint` job (C-70, with the C-79 network and mount assertions, the one-place tag check and the major-drift check, §2.2, §6.1a); image build; the `image` job builds the Postgres and Caddy derivatives, scans both with Trivy, and runs the container tests (settings, `archive_command`, secret-file modes, Caddy headers); provisioning-script `shellcheck` and a dry run in a container
`apps/web/src/routes/**`, Vercel project config	Vercel topology check, three legs plus the env-name check (ARCH §7.6)
`.github/workflows/**`, `scripts/cairn/**`, `.claude/**`	The existing `cairn` job, unchanged. Workflow edits also run an action-pin check (every `uses:` pinned by commit SHA, C-81).
Merge to `main`, any code path	`build.yml` (§4.4), then the staging smoke

Changed paths	Jobs that run
Weekly schedule	The `build` job runs again from `main`, with no cache: base-image patches arrive through the in-place `apt-get upgrade` in the final stage (no patched upstream digest exists for the pinned Node image). Each run pushes `sha-<commit>-<run_id>` and moves no shared tag. A scan-only job also runs Trivy on the digest of the newest published `v1.*` Release (prod is frozen at its tag); red raises an alert, deploys nothing, and is a no-op until a tag exists. The restore-drill reminder issue on the first of the month stays with POLY-100 (§6.7)

Every path a job executes or imports, transitively, triggers that job: `infra/ci/image.sh`, `infra/ci/install-trivy.sh` and `.trivyignore` run the `image` job; the code, scripts and lockfile the DB battery imports or spawns (`packages/config`, `packages/tenancy`, `apps/migrate`, `apps/web/src/ops`, `apps/web/src/main.ts`, `apps/web/src/startup.ts`, `tests/log`, `pnpm-lock.yaml`) run the `db` job, as do `infra/postgres/Dockerfile` and `infra/postgres-major` (the battery builds its Postgres from that file's `FROM` line). The rest of `infra/ci/**` sets only `ci`. A static test follows the battery's import closure and fails when a path escapes the `db` filter.

Web e2e and the harness. The e2e web server (`apps/web/tests/e2e/serve.ts`) uses the DB battery's harness, which Turborepo's affected graph cannot see because the harness is not a workspace package. The `changes` job therefore emits an `e2e` key for `tests/db/helpers`, `tests/db/docker`, `apps/migrate`, `infra/postgres/Dockerfile` and `infra/postgres-major`. The `unit` job runs on it, and the e2e step reads it as `E2E_FORCE`: when it is true the step runs the full `turbo run e2e` even if `@poly/web` is not affected. The aggregate requires `unit` when `e2e` is true, and a full dispatch run sets it. A static test follows the e2e server's import closure and fails when a file outside `apps/web` is neither in a package `@poly/web` depends on nor matched by the `e2e` key.

Generated-output freshness. A committed generated file is registered as a source-to-output pair in `infra/ci/generated.json`, and the `lint` job runs `scripts/cairn/check_dist_freshness.py --ci` over every pair marked `ci`. A `regenerate` pair copies its root to a temporary directory outside the repository, runs the generator there with no stdin and a minimal environment, and fails unless the output is byte-identical to the committed one; history is never read, so clock skew, same-second commits and shallow clones cannot matter. An `ancestry` pair compares commit ancestry and falls back to the same rebuild. The check exits 0 fresh, 1 stale, 2 could not run (a missing registry, generator or git, or a shallow clone for an ancestry pair), and 2 outranks 1. The Drizzle migrations (`packages/db/migrations`) are

the registered pair; the dashboard `dist/` is a built-in pair checked by `/finish-feature`. A feature that commits a generated file registers its pair in the same PR. For migrations the check proves the schema matches the latest snapshot; it does not prove the bytes of an existing migration, which review and the RLS battery cover.

DB battery: parallel and serial groups. The battery runs in one vitest invocation with two projects. `db-parallel` runs every file in `tests/db` except the serial group with file parallelism on, at most 3 workers (4 vCPU less one for Postgres); `db-serial` runs after it, one file at a time: `runner` (role-membership GRANT/REVOKE and cluster-wide `pg_locks` reads), `template-clone` (asserts the login roles' stored password hashes) and `ops-hardening` (a 4.8 s production timeout that is timing-sensitive under contention). The list is `SERIAL` in `tests/db/vitest.config.ts`, and a static test fails when a file outside it writes cluster-wide state. A full run executes the same tests as a serial run (352 at the time of writing, 350 passed and 2 `it.fails`).

Bootstrap serialisation. Every fresh or cloned test database reruns `roles.sql`, which alters cluster-global roles; two sessions doing that at once fail with `tuple concurrently updated`. The harness therefore wraps the psql session of `freshDb()` and `cloneMigrated()` in `withBootstrapLock`, a directory lock (`mkdir` is atomic across processes) inside a per-run directory that globalSetup creates and exports as `POLY_TEST_LOCK_DIR`. It never runs unlocked: an unset directory throws, and a wait past 120 s throws `bootstrap lock timeout`. `roles.sql` itself is unchanged, so what the harness runs is what provisioning runs. The role passwords are set once in globalSetup, before any worker starts.

Area map and the fail-closed rule. The battery's files belong to four areas, `auth`, `tenancy`, `migrate` and `ops`, defined in `infra/ci/changes.mjs` (`DB_AREAS`: each area's test files and the paths its tests import or read). The `changes` job emits `db_auth`, `db_tenancy`, `db_migrate` and `db_ops`; `db` is their OR, so the `db` job's condition and the `app` aggregate are unchanged. On a pull request the battery step runs only the selected areas' files (`changes.mjs --db-select`, read into a bash array, every line re-checked against `^tests/db/[a-z0-9-]+\.test\.ts$`). A path every area shares (`DB_SHARED`: `packages/db`, `apps/migrate`, `packages/config`, the harness, the Postgres image files, the lockfile and root manifests, and the files the `db` job itself executes) runs all four. So does a `db` path that is in no area, and a full dispatch run. An unmapped test file, a file in two areas, a listed file missing from disk, no area on, or a value other than `true` or `false` fails the step. A preflight in the `db` job, before any container starts, runs the area-map, import-closure and harness proofs, and the closure test fails when an area's tests import a file that only another area's paths cover.

pgTAP image build. The harness runs its tests on a pgTAP derivative of the pinned Postgres image, which a fresh runner has to build (base pull plus `apt-get install`). `tests/db/helpers/pgtap-image.mjs`, plain JavaScript on node built-ins, owns the base pin (the `FROM` line of `infra/postgres/Dockerfile`), the image tag and the `docker build` arguments; `pg.ts` imports them and `ci.yml` repeats none of them. The `db` job starts that build in the background, in its own session, before Set up Node, so it overlaps Node, pnpm, install, the preflight and `svelte-kit sync`. The battery step waits for the build's exit-code file (at most 300 s), prints the build log and exit code, and never acts on the code: the harness's own `docker image inspect` decides, and builds the image itself when the tag is absent, so a failed or partial background build costs time and cannot pass a job.

Per-file timing. The battery writes vitest's JSON report, and `job-report.mjs` adds one `ci:db:file` line per test file (file, project, area, seconds, ok) to the `ci-test-runs-db` artifact. The `db` budget is 100 s and stays a warning; a per-file ratchet waits for settled numbers.

Bookkeeping pushes. One rule, in `infra/ci/anchor.mjs`, serves the `changes` job and the `cairn` job. The anchor is the head of the last successful `pull_request` run of `ci.yml` on the branch, from this repository. It counts only if it is in the clone, is an ancestor of the head, and has the PR base as an ancestor, so a merge from `main` or a rebase disqualifies it. When every path from the anchor to the head is a ledger file (`process/cairn/{issues,milestones,majors,archive,reviews}/`, `process/STATE.md`, `process/DECISIONS.md`; the five pinned rulings excluded), `changes` emits every key false, so lint, unit, `db` and `image` skip, and the `cairn` job runs only the cairn tests whose result depends on ledger content plus `scripts/cairn/cairn check`. Any lookup error, missing or empty anchor, failed ancestry or diff check, symlink or gitlink in the diff, or path outside the set gives the normal base-to-head behaviour. Every filter diff is `git diff --no-renames -z --name-only`, so a rename lists both ends and a name is never quoted or split; `changes.mjs` rejects input that is not NUL-terminated. `actions: read` is granted only to the `changes` and `cairn` jobs, and `GH_TOKEN` is set only on the step that runs `anchor.mjs`. The merge still needs a green `app` and `cairn` on the head that merges.

Required checks on `main` stay as they are (`cairn` plus the new `app` aggregate job, which is the single required check for all app jobs so that path-skipped jobs never leave a check "Expected"). The aggregate is red when any `changes` output is missing or not exactly `true` or `false`, or when `db` is not the OR of the area keys. Speed levers, in order: Turborepo remote cache off (no third party), pnpm store cached by lockfile hash, Playwright browsers cached, the DB battery on one Postgres service container with file parallelism and area selection inside the one `db` job. If the inner loop passes 5 minutes, shard the battery across runners before dropping a test.

4.4 Build, scan, attest, deploy (pull-based)

1. **Build once.** `build.yml` runs on every push to `main`: `pnpm install --frozen-lockfile` with lifecycle scripts blocked (C-80), build the one `adapter-node` image, push to GHCR tagged with the git sha, record the digest.
2. **Scan and attest.** Trivy on the image (fail on fixable critical or high and on embedded secrets, C-82); CycloneDX SBOM; `actions/attest-build-provenance` for the digest. The same step builds and attests the custom Caddy image and the tools image.
3. **The runtime image carries no package manager.** `npm`, `npx`, `corepack`, `pnpm` and `yarn` do not resolve on `PATH`, `/usr/local/lib/node_modules` is absent, and the image runs as `node` (non-root; apt and dpkg remain in the slim base and are acceptable only because of that). `infra/ci/image.sh check` asserts it on the built image in the PR `image` job and in `build.yml`, before the scan. apt uses only the base image's signed sources and only `update` and `upgrade`.
4. **Release.** When `/merge-pr` pushes a signed `v1.*` tag, `release.yml` looks up the digest already built for that commit (never rebuilds), checks that its attestation exists, and publishes a GitHub Release whose body is one line, `digest: sha256:…`. If no image exists for the tagged commit the workflow fails and nothing deploys. The lookup finds a digest only while that commit's `image-<commit>` artifact exists, which is 90 days after its `main` build (the ceiling GitHub allows on a public repository). The weekly rebuild refreshes the artifact for `main`'s head. To release an older commit, merge a new commit and tag that; re-running `build.yml` builds `main`'s head, never an old commit.
5. **The box polls.** A systemd timer runs `polycarpic-deploy` every 5 minutes (with 60 s jitter) under `flock`, so two runs never overlap. Staging: newest `main` digest from GHCR. Prod: newest `v1.*` release from the Releases API.
6. **Verify, then act.** If the digest equals `current-digest`, exit. Otherwise: pull by digest; `gh attestation verify --repo richmosko/polycarpic --signer-workflow richmosko/polycarpic/.github/workflows/build.yml --source-ref refs/heads/main --deny-self-hosted-runners`; **for prod also** `git verify-tag` against `/etc/polycarpic/allowed_signers` and a check that the tag's commit is the one the digest was built from (the image carries its commit as a label, also covered by the attestation). Any failure: log, alert, stop. The box never runs an unverified image.
7. **Extract the ops bundle** (compose files, Caddyfile, scripts) from the verified image into `/opt/polycarpic/releases/<digest>/`, so the compose definitions are covered by the same attestation as the code.
8. **Pre-migration check**, then the migration (§4.5), `docker compose up -d` for the project, then poll `/api/v1/health` for up to 120 seconds.

9. **Result.** Healthy: write `current-digest`, log the deploy, ping the monitor. Unhealthy: automatic rollback (§4.6) and an alert.

Not changing during a deploy: secrets (placed by you, never by CI), the deploy script itself and the timer unit. Those change by an operator step (`polycarpic-ops self-update`, §8.7) that runs the same verification on a newer release, because a script that rewrites itself from the network is the supply-chain path this design exists to close.

4.5 Migration deployment mechanism

The tool is the one the POLY-76 spike ratifies (Drizzle Kit recommended, ARCH §6.2). Whichever tool wins, the mechanism below is fixed because it follows from the spike's constraints.

- **One-off container, not app startup.** The `migrate` service (same image, `migrate` entrypoint) connects as `poly_migrator`, the only login that may run DDL, and runs each file as `set role poly_owner … reset role` so objects are owned by the NOLOGIN owner and FORCE RLS holds. The migrate module is the one place a session-level `SET ROLE` is allowed (a dedicated connection, closed after the run, never pooled; ARCH §2.1); a custom file that switches to another definer owner ends on `SET ROLE poly_owner`. Drizzle Kit is the runner (POLY-76, ARCH §6.2). `web` and `worker` never get the migrator password and refuse to start if the schema version is behind the image.
- **Who owns what in the database (POLY-76, ARCH §6.2).** `provision.sh` makes `poly_owner` the owner of the database; `poly_owner` creates the `drizzle` tracking schema and `drizzle.__drizzle_migrations` (it is the active role when the first file runs). `poly_owner` is `NOLOGIN NOBYPASSRLS` and `poly_migrator` is its only member, granted `WITH INHERIT FALSE, SET TRUE` so it holds none of `poly_owner`'s privileges until it runs `SET ROLE` (the run is proven to end as `poly_owner` because only `poly_owner` may insert tracking rows; with inherited privileges a `RESET ROLE` would pass as `poly_migrator`; SECURITY C-85). The grant carries no `ADMIN` option, and no role other than `poly_owner` holds `INSERT`, `UPDATE`, `DELETE` or `TRUNCATE` on the tracking table. `poly_migrator` owns no object (SECURITY C-85; a catalog test over `pg_class`, `pg_proc`, `pg_namespace`, `pg_type` is POLY-83's). `poly_migrator` is proposed to hold **no database `CREATE` of its own**: *target, unmeasured*, because the spike granted both; POLY-83 measures it, and until then provisioning must not be written to depend on either answer. `poly_app` and `poly_worker` get `USAGE` on `drizzle` and `SELECT` on the tracking table, nothing more; that is all the "schema behind image" start check needs. Neon gets the same grants from the first migration.
- **Definers and policies.** SECURITY DEFINER functions are created owned by their role (`poly_authz`, `poly_purge`, and so on) and policies target roles with `to:`; the DB battery

proves both through the same runner on every PR, so a migration that works only for a superuser cannot reach the box.

- **Order.** `check-config` (secrets present, `TLS_MODE`, `PLAID_ENV`) → `migrate --plan` (lists pending files, exits) → if any are pending, a **pre-migration backup** (the `backup` container runs `pg_dump --format=custom` as `poly_backup` through the same age pipe as the nightly set into `spool/premigration/`; the `uploader` ships it like any backup; it stays in the spool 7 days; no other dump path exists, C-79) → `migrate` (takes `pg_advisory_lock` on its connection before running, so a second concurrent run waits and applies nothing; and refuses, as does `migrate --plan`, when the applied rows in `drizzle.__drizzle_migrations` are not a hash-for-hash prefix of the journal) → `up -d`.
- **Failure.** All pending migrations run in *one* transaction (drizzle-orm 0.45.4 behaviour, read from source), so a failing file leaves the tracking table and the schema as they were. Consequence: no `CREATE INDEX CONCURRENTLY` in a migration (a CI check refuses it). CI also refuses a journal whose `when` values are not strictly increasing, because the runner would silently skip a migration older than one already applied (ARCH §6.2, POLY-83). Unmeasured until POLY-83 and POLY-100 run it. The deploy stops, the old containers keep running on the old digest, and an alert is sent. Nothing is rolled forward by hand.
- **Expand and contract.** A release only ever adds (columns, tables, nullable fields, new functions). A removal ships at least one release after the code that used the thing is gone. Therefore the previous release's code always runs on the new schema, which is what makes rollback a one-file change.
- **Vercel and Neon.** The same `migrate` entrypoint runs in the Vercel build step against Neon's **direct** endpoint; the runtime uses the pooled one (§1.4, D-6).

4.6 Rollback

- **Automatic.** If health does not pass in 120 seconds after `up -d`, the script points `desired-digest` back at `current-digest`, restarts that release's containers from its own ops bundle, writes the failure to `deploy.log` and alerts. The failed digest is recorded in `/opt/polycarpic/quarantine` and skipped by the poll until a newer one appears.
- **Manual, one command.** `polycarpic-ops rollback` reverts to the previous digest recorded in `deploy.log` without contacting GitHub, then keeps the poll from re-deploying the newer release by quarantining it. §8.6 has the steps.
- **Schema.** A code rollback never needs a schema rollback (expand-contract). A schema-level undo is the restore path (§6.8), starting from the pre-migration backup, restored off the box (§6.4).

- **Brief interruption.** `up -d` recreates `web`, so for a few seconds Caddy has no healthy upstream; Caddy retries for 10 seconds (`lb_try_duration`) which hides it for most requests. ARCH §8.1's "keeps routing to the last healthy containers" is not literally achievable with a plain recreate on one box; v1 accepts the few seconds.

4.7 Vercel path

The operator deploys a verified signed tag with `infra/vercel/deploy.sh` (D-11), after the topology checks of ARCH §7.6 which run in GitHub Actions on every PR touching routes or project config. The Vercel environment-variable names check uses a read-scoped Vercel token held as a GitHub environment secret with required reviewers; it reads names, never values. No deployment credential for Vercel lives in GitHub. A failed migration step fails the build and Vercel keeps serving the previous deployment.

5. Update & Patch Schedule

Automatic where an unattended failure is cheap, manual in a fixed window where it is not. Times are the operator's local time.

What	How	When	Who
Ubuntu security updates (kernel, libc, openssl, sshd)	`unattended-upgrades`, security pocket only. Automatic reboot at 04:30 if a reboot is needed. Containers return on their own (`restart: unless-stopped`, `live-restore`). Home box: PCR 7 only keeps the TPM unlock valid across kernel updates (§3.5).	Daily check; reboot in the window	Automatic. The outside monitor alerts if the box is down for more than 15 minutes after 04:30.
Other apt packages, Docker Engine, Compose plugin	`apt full-upgrade` by hand, in the window, one box at a time (§8.2)	First Sunday of the month	Operator
Application base images (Node, Postgres minor, Caddy), OS layers inside images	The weekly scheduled rebuild of `main` picks up patched Debian layers through the in-place upgrade; Renovate proposes the pinned-digest bump when upstream ships one. Staging deploys automatically.	Weekly rebuild Monday (cron 05:17 UTC); Renovate weekly	CI. Merges are reviewed; minor and patch bumps may auto-merge only when CI is green and `minimumReleaseAge` of 3 days has passed (C-80). Majors never auto-merge.
A prod release	A signed tag cut by `/merge-pr` from a staging-proven `main`	When a feature ships, and at least monthly while any base-image or dependency fix is pending	Team-lead via `/merge-pr`; the box deploys itself

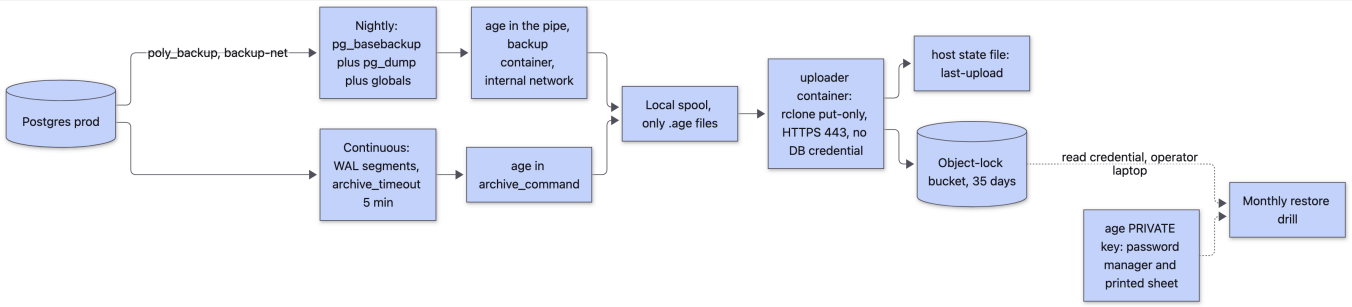
What	How	When	Who
Vulnerability response in a deployed dependency or image	Known-exploited or critical with a fix: patch release within 48 hours. High with a fix: within 7 days. Anything else: next monthly release. Compromised dependency: SECURITY RB-6.	As found by Trivy, audit, or advisories	Team-lead dispatches; the box deploys
`better-auth`, `@better-auth/passkey`	Exact version pins (no ranges). A bump is its own PR and must pass the routing-pool test and the pinned-option checks of SECURITY A-33.	Reviewed when Renovate proposes; at the latest each quarter	Backend lead with SecEng review
Postgres minor	Arrives with the image digest bump, which edits the one `FROM postgres:17.N@sha256:…` line (§6.1a); runs the DB battery first	With the weekly rebuild	CI
Postgres major	Never in place. Dump and restore into a new volume as a drill (§6.7) on staging first; scheduled once a year or when the major nears end of life, and Neon's major is changed in step.	Evaluated each October	Operator with architect
Operator secrets	Rotation schedule in §8	Annual review; on suspicion immediately	Operator
GitHub read token	Expires every 90 days	Renew in week 11	Operator
Private CA intermediate (private-CA mode)	2-year validity	Replace 60 days before expiry	Operator

Restart-after-change pattern. The repository already has the rule for the tooling receiver: a merge changes what should be running, and whatever serves it is restarted by the merge process, not remembered by a person (CLAUDE.md, `/merge-pr`). The deploy script applies the same rule to the app: the poll compares what is running with what should be, and a deploy always ends with a health check, never with "the command returned".

6. Disaster Recovery & Monitoring

Resolves ARCH OQ-9 detail and OQ-26; decided in §9 (D-1 to D-4). In plain terms: backups are locked with a key that never lives on the server, stored where the server cannot delete them, and proven by restoring them somewhere else once a month.

6.1 What is backed up and how



- **Nightly (02:15).** The `backup` container runs three artifacts as `poly_backup` (SECURITY C-79), each piped through `age` to the recipient public key *as it is taken*, so plaintext never reaches the spool: (1) `pg_basebackup -Ft -X none -D -`, a tar of the cluster, the base for point-in-time recovery (`-X none` because `pg_basebackup` cannot stream WAL in tar mode to stdout; the archived WAL covers the backup window, so there is also no replication slot to leak, unmeasured until OQ-INFRA-11); (2) `pg_dump --format=custom` of the database, the portable fallback that restores onto any Postgres of the same or a newer major; (3) `pg_dumpall --globals-only --no-role-passwords`, which carries the roles (`poly_owner`, `poly_app` and the rest) that a per-database dump omits, without password hashes (restored logins get new passwords from the password manager; the drill sets them). A restore without (3) comes up without its RLS roles, so the drill checks for it. The job writes `last-dump` and a line in `jobs.log`, then starts the `uploader`.
- **Continuous.** Postgres's `archive_command` encrypts each WAL segment with `age` (public key from the mounted config) into `spool/wal/<segment>.age`, writing a temporary name and renaming, and returns failure if the file already exists; the `uploader` runs every minute and ships what it finds. `archive_timeout = 300` forces a segment at least every 5 minutes while the database is active, so the worst case loss is about 5 minutes plus upload delay, inside the 15-minute RPO. If the spool volume fills, archiving fails and Postgres keeps WAL: the 2-hour spool alert and the disk alert (§6.5) are what catch it.
- **Corrections to ARCH §7.2.** WAL replay needs a physical base backup; a `pg_dump` alone cannot be the base for WAL. The nightly set above adds `pg_basebackup`. The base backup cannot use a temporary slot (`-X stream`) in tar-to-stdout mode, so the set uses `-X none`

and relies on the archive. And a WAL segment must be encrypted before it reaches the spool, which needs `age` in the Postgres image. Physical restore needs the same Postgres major, so the image's major is pinned and recorded in each backup's name.

- **Not backed up, by design:** the credential keyring (SECURITY §6.4: losing it means relinking), the secrets directory, staging's database, the age private key. Anything in the secrets directory is recreated from the password manager and provider consoles (§8.9).
- **Where.** A **Backblaze B2** bucket (EU region) with Object Lock in compliance mode and a 35-day default retention, reached through B2's S3-compatible endpoint with `rclone` (`type = s3`, `provider = Other`, `endpoint = https://s3.<region>.backblazeb2.com`, `no_check_bucket = true` because the key is bucket-restricted), written by a put-only credential over HTTPS 443 (works on every target, including the home box's outbound allowlist). The store, not the box, enforces that nothing can be deleted or overwritten before 35 days. The box credential cannot shorten retention or delete versions. The Storage Box is not the default (D-3): a Storage Box "append-only" mode is unmeasured and its SFTP port is blocked on the home box. A Storage Box remains a documented fallback on the Hetzner target once its cannot-delete drill passes. Hetzner Object Storage stays a noted alternative pending verification of its compliance-mode lock; the Storage Box is the fallback only. B2 specifics *(unmeasured until the first drill, OQ-INFRA-4)*: the put-only application key holds the `writeFiles` capability on this bucket only (plus `listBuckets` if rclone needs it) and none of `deleteFiles`, `bypassGovernance`, `writeFileRetentions`, `writeBucketRetentions`; the read key holds `listFiles` and `readFiles` only; the account's master key is never put on the box or used for backups. A lifecycle rule hides objects 36 days after upload and deletes the hidden version a day later, so storage does not grow without bound while the lock still guarantees 35 full days. **Caps (C-73 residual).** In B2 Caps & Alerts set a storage cap (start 10 GB) with an email at 75 percent, and a transaction alert; leave downloads uncapped so a break-glass restore is never blocked by a cap (B2 specifics unmeasured, OQ-INFRA-4). **Upload step.** Only the `uploader` talks to the bucket: for each spool file it uploads, writes the `.sha256` sibling, lists the object and compares, and only then deletes the spool copy and writes `last-upload` and pings the monitor; any mismatch pings `/fail` and leaves the file.

6.1a The Postgres image and minor

ARCH fixes the major (17, in `infra/postgres-major`); the image and minor are INFRA's. **Image:** the official `postgres` image, tag `17.10`, the minor the POLY-76 spike measured. Prod and staging run the `age` derivative (§2.2) whose single `FROM postgres:17.10@sha256:…` line is the only place the minor and digest are written. **CI** starts its Postgres service from the same tag and digest (read from that line, not retyped), and the **backup container** uses the same image so `pg_basebackup`, `pg_dump` and a restore drill run the server's own minor. **Neon** is created on

Postgres 17; Neon applies its own minors and they are not ours to pin. The digest is filled in at the first build (unmeasured until then).

Who bumps a minor. Renovate proposes the new tag and digest on that one line (weekly, §5); CI runs the DB battery against it first; the devops-engineer reviews and merges. Minors stay inside 17 and need no restore drill, because a physical restore needs only the same major. A major is never a bump (§5, §6.7). **Drift check:** a lint job in the infra lint (POLY-95) fails when the major on the `FROM` line differs from `infra/postgres-major`, so the file and the image cannot disagree (ARCH §6).

6.2 Naming and integrity

Objects are named `<instance>/<pgmajor>/<yyyy>/<mm>/<dd>/<kind>-<utc-timestamp>.age` with a `.sha256` sibling written after the upload. The backup job fails, and alerts, if a listed object does not match its recorded checksum after upload. The age private key never appears in any path or log.

6.3 Retention (SECURITY R-5)

Everything in the bucket expires 35 days after it is written and the store allows nothing earlier. So a purged entity's data can remain in backups for at most **35 days** after the purge; the purge confirmation screen should say "up to 35 days". There is no long-term archive tier in v1: if you need a yearly keepsake, export from the app (the export carries the hash-chain head) and keep it yourself. Lifecycle expiry is a bucket rule, so no deletion credential exists anywhere.

6.4 RPO and RTO by target

Target	RPO	RTO	How
Hetzner VPS	15 minutes (WAL). Fallback: 24 hours (nightly dump).	4 hours for total loss of the server: provision from the same provisioning files, restore, redeploy the last release. 60 minutes for a bad migration that changed data wrongly, box intact (§4.6; restored off the box, below). A failed migration file needs no restore at all: it ran in a transaction (§4.5).	§8.10, rehearsed monthly in part (§6.7)
Local box	15 minutes (same WAL path)	1 day to repair or replace the box if you hold spare hardware or can buy it; 4 hours to a **temporary Hetzner VPS** restored from the same bucket, then move back	Restore drill identical

Target	RPO	RTO	How
		(§8.10). Because the box is the secondary target, rented capacity is the break-glass.	to Hetzner
Vercel + Neon	Neon's restore window (unmeasured; plan-dependent) plus weekly export: worst case 7 days	4 hours: new Neon project, restore export, repoint env vars	§6.6

The bad-migration restore, as it actually works (answers SECURITY §12.3, C-79)

- **The constraint.** The pre-migration backup is encrypted to the age public key like every other backup, and the private key is on no box. So the decrypt cannot run on the box, and ARCH's 30 minutes (which assumed it could) does not hold.
- **The path.** The encrypted file is on the box's spool for 7 days and in the bucket. On the laptop: `infra/restore.sh --premigration --from-box polyops@<host>` copies the newest encrypted file from the spool (a few MB to a few hundred MB at household volume, unmeasured), decrypts it with the age private key from the password manager, and sends the plain dump to `/srv/polycarpic/restore/` on the box's encrypted volume. Then `box$ sudo polycarpic-ops restore-apply prod --dump --into-scratch` restores it into a scratch database next to prod, for you to compare, and a second command swaps or copies back what you decide. The plain copy is deleted afterwards on both machines.
- **Time (unmeasured until the drill).** About 10 minutes to reach the laptop and key, 10 to copy and decrypt, 10 to restore into scratch, 20 to compare and decide and apply: **60 minutes**. The 30-minute figure is withdrawn. If the laptop is unavailable, the key is on the recovery sheet and any machine with `age` will do (§8.10 step 3).
- **Rejected: an unencrypted copy kept on the LUKS spool.** It would put a plaintext copy of the whole ledger, including BetterAuth's password hashes and session tokens, on the same box as the application, which defeats the property the age scheme exists to give (a captured running box yields no readable backup). SECURITY would have to rule on it. The pre-migration restore takes 60 minutes, run off the box (§6.4, D-14).

6.5 Monitoring, heartbeat and alert routing

- **Two independent alert paths**, so one failure never silences both. Path A: the app and host scripts email you through Resend (SECURITY §9 list, plus backup, deploy and certificate events). Path B: an outside monitor that does not depend on the box or Resend: **Healthchecks.io**, free tier, a dead-man's-switch service. Each box pings it, and silence past a check's grace period emails you. It holds no data: a ping carries no body, only the fact that it arrived.
- **What pings Path B.** (1) Every 5 minutes, only if local `/api/v1/health` is OK *and* the served certificate's issuer matches `TLS_MODE` (C-76); (2) after each upload the `uploader` has verified against its checksum; (3) after each successful deploy poll. Grace: 15 minutes, 26 hours and 20 minutes respectively. The ping carries no data. Healthchecks.io only listens; it does not probe your URL. So before the 5-minute ping the host script makes the same request a browser would (the Hetzner box requests its public name through public DNS, the home box its LAN name) and skips the ping if that fails, which covers Caddy and DNS faults a local health check would miss. A script that detects a failure on its own (a failed backup, a rolled-back deploy) sends the ping to the check's `/fail` address so the alert arrives at once instead of at the grace deadline.
- **Healthchecks.io setup (once).** Create the account with two-step sign-in and your own address as the alert email. In one project create six checks: `polycarpic-hetzner-alive`, `-backup`, `-deploy` and, for the home box, `polycarpic-home-alive`, `-backup`, `-deploy`, each with the period and grace given above and a name that tells you which box it is. Copy the project's *ping key*; the secret `heartbeat_url` on each box is `https://hc-ping.com/<ping-key>/` and the scripts append the check's slug. Leave auto-creation of checks off. The monthly drill and the 90-day GitHub token get a check each too (period 35 days and 90 days, pinged by you at the end of the task, §8.2 and §8.4). Free-tier check count and grace limits are unmeasured (OQ-INFRA-9).
- **Health endpoint exposure (ARCH §7.4).** Public `/api/v1/health` returns only `ok` or not. The detail ARCH §7.4 lists (database connectivity, applied schema version against the image's, last job runs) is served by the status listener inside `web`, bound to 127.0.0.1 in the container, never published and never routed by Caddy, and read by the host scripts with `docker compose exec web` until POLY-81 moves it into an `admin` service. **Backup age is not app state:** the `uploader` writes `/var/lib/polycarpic/state/last-upload` (a timestamp and the object names) only after a verified upload, and `polycarpic-ops status`, the 26-hour alert and the backup heartbeat read that file. A stranger learns nothing about backup cadence, and a dead `web` does not hide a healthy backup.
- **Alerts that need no new service:** spool older than 2 hours; backup older than 26 hours (from `last-upload`); any non-temporary row in `pg_replication_slots` (the 5-minute ping is also skipped, C-79); WAL kept on disk by a slot over half of the 2 GB cap; a `poly_backup`

connection in the last 24 hours that no `jobs.log` line accounts for (`status` lists them); B2 storage at 75 percent of its cap; certificate under 14 days; disk over 80 percent; deploy failed or rolled back; a reboot pending (`/var/run/reboot-required` older than 3 days on Hetzner; older than 7 days on the home box, where automatic reboot stays off until the kernel-update reboot test passes, §3.5, SECURITY C-72), checked daily by a timer and sent through both alert paths; GitHub token under 14 days; ledger hash-chain break (SEV1) or a verifier silent for 8 days. All thresholds are provisional and tuned after POLY-M1.

- **Logs.** JSON to stdout, Docker `json-file` rotation, 30 days on the box (SECURITY §9). The operator's query surface is `docker compose logs` and `journalctl`. No log service in v1.
- **The dashboard the lead can point to:** the monitor's check list (green or red per box) plus `polycarpic-ops status`, which prints the running digest, last deploy, last backup, certificate days left and disk use on one screen.

6.6 Vercel + Neon backups (answers OQ-26)

- A cron function cannot run `pg_dump` or ship WAL, so the Compose mechanism does not apply.
- **Default (v1.0, no new trust):** rely on Neon's point-in-time restore for the plan in use (window unmeasured; read it on the plan page at deploy time and record it here), and run `polycarpic-ops neon-export` **from the operator's laptop weekly**: `pg_dump` through a read-only role, age-encrypted to the same public key, uploaded with the put-only credential to the same bucket. RPO is then up to 7 days beyond Neon's window, stated honestly.
- **The weekly operator-run export stands; no database credential is held in GitHub.** Revisit with SecEng only if real data ever lands on the tertiary target; a nightly Actions export would make GitHub a holder of a ledger-read credential (TB-12) and needs a SecEng ruling first (OQ-INFRA-1, S-6).
- Restore: create a Neon project, run `polycarpic-ops restore --dump` from the laptop (§8.10), repoint the Vercel variables, redeploy.

6.7 The restore drill (a GA gate)

Monthly, scripted, and **run off the box**: your laptop or a throwaway server, never the VPS, because the age private key must never reach it (ARCH §7.2). The first passing drill against real backup objects is a gate for v1.0; a drill that has not run in 45 days raises an issue and an email.

1. List the bucket with the read credential; take the newest base backup and WAL.
2. Decrypt with the age private key from the password manager. Assert the globals file contains no password hash (`--no-role-passwords`); the drill then sets passwords on the restored

roles.

3. Start a scratch Postgres at the recorded major; restore the globals, then recover to the latest WAL; separately restore the `pg_dump` into a second scratch instance.
4. Assert: the trial balance of every entity sums to zero; the per-entity hash-chain verifier passes (C-24); the roles from the globals file exist; the latest journal entry is within 20 minutes of the drill's start for the WAL path; the row count of a `FORCE ROW LEVEL SECURITY` table restored from the `pg_dump` equals the count in the base-backup restore (proof that `BYPASSRLS` dumped it whole, C-79).
5. Assert, using the box credential file copied for the drill, that **deleting an existing object fails**, so does **overwriting an object's version**, shortening its retention, and changing the bucket's default retention (C-73's second test).
6. Record date, backup names, timings and pass or fail in `process/cairn/reviews/restore-drill-<date>.md` via the lead (not edited by hand on the box). Destroy the scratch instances.

6.8 Recovery procedures

Event	Procedure
Bad migration or bad release, box intact	§8.6 rollback. If data changed wrongly, restore the pre-migration backup into a scratch database, compare, and decide; do not overwrite prod without the drill checks.
Box lost (fire, failed disk, dead provider)	§8.10: new server from the same files, restore from the bucket, redeploy the last release, re-enter secrets from the password manager, verify, repoint DNS.
Suspected compromise of the box or of an	SECURITY RB-1 and RB-2. Commands in §8.11.

Event	Procedure
operator secret	
Stolen home box (RB-2 variant)	The disk stays locked only while the thief cannot boot it; assume it boots (R-21). Within the hour: (1) from the registrar/DNS console, revoke the zone token; (2) revoke every bank connection at its provider (Plaid `/item/remove`, SimpleFIN bridge, broker OAuth); (3) rotate the age keypair is *not* needed, but rotate the bucket put-credential at the provider and the Resend key; (4) rotate the BetterAuth secret and every OAuth client secret; (5) tell the household; (6) stand up a replacement (§8.10) from backups taken before the theft and relink every connection; (7) file the postmortem within 5 business days (SECURITY §10). The private CA root, if used, was never on the box; replace the intermediate.

7. Deployment Runbook

Written for someone who has not run a server before. Every step has the command, what you should see, and what to do if you do not. Do the steps in order and do not skip the checks. Commands marked `laptop$` *run on your own computer,* `box$` *on the server.* `polycarpic-ops` *is the small operator tool the repository ships under* `infra/`*; its subcommands are specified here and built under POLY-M1.*

> **Before you start.** No real books exist before v1.0. Until then, do §7.1 with *staging only* to learn the process with fake data; the prod steps are marked *(from v1.0)*. If any step fails twice, stop and ask. Do not improvise on a box that will hold real data.

7.0 What you need before step 1

Item	Why	Cost (unmeasured)
A password manager you trust, with an emergency kit	Holds every key and recovery code	Whatever you use now
A domain name you own, and an account where you can edit its DNS	Names, certificates, sign-in providers	About 10 to 15 USD a year
A Hetzner Cloud account (card required, ID check possible)	The primary server	Roughly 10 to 15 EUR a month for the server, volume and backup
A Backblaze B2 account (Object Lock bucket)	Backups	Cents a month at this size
A Resend account and a verified sending domain	Sign-in and security email	Free tier
A Healthchecks.io account	Tells you when a box goes quiet; see §6.5	Free tier (limits unmeasured)
Your GitHub login, and the `gh` tool signed in	Pulling releases and checking they are genuine	Free
A Mac or Linux laptop with a terminal	Everything marked `laptop$`	—

Operator-account checklist (SECURITY C-78). Before step 1, and re-read in every monthly window (§8.2), because `provision-check` cannot see any of it. Each account that can reach a target has (a) two-step sign-in with its codes in the password manager, (b) its own password, (c) your own address for recovery. Tick each: Hetzner, Cloudflare, Backblaze, GitHub, Healthchecks.io, Resend, the domain registrar, and Vercel and Neon if you use them. Also: the domain has the registrar's transfer lock on. The Hetzner API token is short-lived by design (§7.1 step 7).

7.1 Primary target: Hetzner VPS

1. **Install the tools on your laptop.**

   ```
   laptop$ brew install age hcloud gh git rclone nmap
   laptop$ age --version && hcloud version && gh --version | head -1
   ```

   Expect: a version line from each of the three. If `command not found`, the install failed: read the error from `brew`; on Linux use your package manager for the same names.

2. **Create your SSH key** (your proof of identity to the server).

   ```
   laptop$ ssh-keygen -t ed25519 -C "polycarpic-operator" -f ~/.ssh/polycarpic_ed25519
   laptop$ cat ~/.ssh/polycarpic_ed25519.pub
   ```

   Expect: it asks for a passphrase: set one, and store it in the password manager. The second command prints one line starting `ssh-ed25519`. Never share the file without `.pub`.

3. **Create the backup key** (this is the most important key you will ever make).

   ```
   laptop$ age-keygen -o polycarpic-backup.key
   Public key: age1...
   ```

   Expect: it prints `Public key: age1…`. Copy the public key line into a note called `backup-public-key`. Open `polycarpic-backup.key`, copy the whole file into the password manager as a secure note, and print one copy for the recovery sheet (§3.6). Then `rm polycarpic-backup.key`. If you lose this key, **no backup can ever be opened**; if someone else gets it, they can read every backup. Keep it off the server forever.

4. **Create the backup bucket and two keys at Backblaze B2.** Sign up at backblaze.com (turn on two-step sign-in and save the codes in the password manager). In B2 Cloud Storage → Buckets → Create a Bucket: name `polycarpic-<yourname>-backups`, **Private**, encryption off

(the files are already age-encrypted), and **Object Lock: Enable**, which can only be done at creation. Then open the bucket's Object Lock settings: default retention **Compliance mode, 35 days**. Add a lifecycle rule: hide 36 days after upload, delete 1 day after hiding. Under App Keys → Add a New Application Key create two keys, each **restricted to this bucket**: `polycarpic-put` with only the *write* permission (B2 shows this as "Write Only" access), and `polycarpic-read` with *Read Only*. Copy each key ID and key once. Note the bucket's S3 endpoint (shown on the bucket page, like `s3.eu-central-003.backblazeb2.com`). Under Caps & Alerts set a storage cap (start 10 GB) with an email alert at 75 percent; leave downloads uncapped so a restore is never blocked (C-73).

Expect: you now hold two key pairs. Store both in the password manager, labelled `b2-put-only (goes on the server)` and `b2-read (laptop only)`, together with the endpoint. Check: `rclone lsd` or `rclone ls` with the read key lists the empty bucket without error. If the bucket page shows Object Lock as "disabled", it was created without it and cannot be fixed: delete the empty bucket and create it again. If only "governance" mode is offered, stop and ask: governance mode can be bypassed by a key holding that permission, so it is not what is required.

5. **Hetzner project and API token.** In the Hetzner Cloud console create a project named `polycarpic`, then Security → API tokens → Generate (Read & Write). Copy the token once.

```
laptop$ hcloud context create polycarpic
(paste the token when asked)
laptop$ hcloud server list
```

Expect: an empty table, no error. If it says `unauthorized`, the token was copied wrong: generate a new one.

6. **Get the repository and read the plan.**

```
laptop$ git clone git@github.com:richmosko/polycarpic.git && cd polycarpic
laptop$ git tag -v v1.0.0     # from v1.0; before then use main
```

Expect (from v1.0): `Good "git" signature for …`. If it says `BAD` or shows no signature, do not continue.

7. **Create the server, firewall and volume.**

```
laptop$ infra/hetzner/create.sh --name polycarpic --location fsn1 \
          --ssh-key ~/.ssh/polycarpic_ed25519.pub
```

Expect: it prints the server's IPv4 and IPv6 addresses and ends with `created: server, firewall (22,80,443 in), volume 20 GB`. It is safe to run again; it skips what exists. If it says the location has no stock, use `nbg1` (Nuremberg): same country, same prices, and the backup bucket is unaffected. If it prints anything about opening ports other than 22, 80, 443, stop.

Then remove the token (SECURITY C-78). It can rebuild the server, move the volume or open the rescue console, so it does not stay on your laptop:

```
laptop$ hcloud context delete polycarpic
laptop$ ls ~/.config/hcloud
```

Expect: the context is deleted and the directory holds no token. Then, in the Hetzner console, Security → API tokens, delete the token. The next run of `create.sh` (or §8.10) starts with a freshly generated one.

8. **Point your DNS at it.** If the domain's name servers are not yet Cloudflare's: add the domain to a free Cloudflare account, copy the two name servers it gives you into the registrar's "name servers" setting (the registrar stays the registrar), and wait until Cloudflare shows the zone Active (minutes to a day). Then in Cloudflare DNS create `A` and `AAAA` records: `app` and `staging` → the two addresses. TTL 300, proxy status **DNS only** (grey cloud; an orange cloud would put Cloudflare in front of the server and break the design).

   ```
   laptop$ dig +short app.example.com A
   laptop$ dig +short app.example.com AAAA
   ```

   Expect: the server's addresses within a few minutes. If empty after 30 minutes, the records are on the wrong zone or have a typo.

9. **First login.**

   ```
   laptop$ ssh -i ~/.ssh/polycarpic_ed25519 polyops@app.example.com
   box$ cloud-init status --wait
   ```

   Expect: the first time, a question about the host fingerprint: compare it with the one in the Hetzner console (server → Console shows it in the boot log) before typing `yes`. Then `status: done`. If `status: error`, run `sudo cat /var/log/cloud-init-output.log | tail -50` and ask; do not continue. If login is refused, the key in step 7 was not the one in step 2.

10. **Hand the box its three public facts** (nothing secret yet).

```
laptop$ scp -i ~/.ssh/polycarpic_ed25519 \
          ~/.config/polycarpic/allowed_signers \
          polyops@app.example.com:/tmp/allowed_signers
laptop$ ssh polyops@app.example.com "echo 'age1...yourpublickey' | sudo tee /etc/polyc
```

Expect: the file copy shows 100 percent. `allowed_signers` is your own copy of the release-signing public keys (make it from the key `/merge-pr` signs with, once; keep it in the password manager). It must not come from the repository (§3.3).

11. **Provision the box.**

```
box$ sudo polycarpic-ops provision --target hetzner --domain example.com
box$ sudo polycarpic-ops provision-check
```

Expect: `provision` prints each stage and ends `done`; `provision-check` ends `ALL CHECKS PASSED`, listing: SSH key-only and no root login; unattended upgrades on; Docker installed; the data volume encrypted and mounted at `/srv/polycarpic`; swap off; secrets directory 0700 and empty of anything but generated files; deploy timer enabled; the running `sshd -T` output (not the config file) shows key-only login, no root, no forwarding; and the C-79 set: from `web` and from `uploader` a `poly_backup` login (SQL and replication) is rejected, from `backup` it succeeds, `backup` cannot reach the internet, `uploader` has no `db_password_backup` and no route to Postgres, the spool holds only `.age` files, and the slot and WAL settings are in place. Any line that says `FAIL` names the fix; rerun `provision` and then the check. Do not continue past a FAIL.

12. **Enter the provider secrets.** For each secret in the list below, run the command and paste the value when prompted (nothing is shown as you type).

```
box$ sudo polycarpic-ops set-secret resend_api_key
box$ sudo polycarpic-ops set-secret backup_store_credential   # the PUT-ONLY pair (the
box$ sudo polycarpic-ops set-secret heartbeat_url
box$ sudo polycarpic-ops set-secret oauth_google_secret      # and the other providers
box$ sudo polycarpic-ops list-secrets
```

Expect: `list-secrets` prints names, sizes and permissions (`0440 root:<gid>`), never values. For staging add `--staging` to each command and use sandbox values only. Missing a required one is not a problem now: the first start tells you which. You do not enter any `db_password_*`: `provision` generated them, `db_password_backup` included. Never paste a secret into a chat, an issue or a file in the repository.

13. **First deploy of staging.**

```
box$ sudo polycarpic-ops first-deploy staging
box$ sudo polycarpic-ops init-keyring --staging
```

Expect: it pulls the newest `main` image, prints `attestation verified`, runs the config check and the migration, starts the containers and ends `healthy`. `init-keyring` prints the staging keyring once; staging's keyring may be lost freely (relink sandbox items), so do not worry about saving it. If it prints `attestation verification FAILED`: **stop**; do not override; check that `gh` on the box is signed in with the read token (`polycarpic-ops set-secret github_read_token`) and that the image really came from this repository.

14. **Prod (from v1.0): the keyring, then the first deploy, with the door closed.**

```
box$ sudo polycarpic-ops first-deploy prod
box$ sudo polycarpic-ops init-keyring
```

Expect: `first-deploy prod` requires the signed tag check to pass and ends `healthy (maintenance page is serving)`: Caddy shows a static "not open yet" page for the prod name until you open it in step 17. `init-keyring` prints the prod keyring **once**: copy the whole block into the password manager (as a secure note separate from the age key) and the recovery sheet, then confirm with `polycarpic-ops keyring-fingerprint` that what you saved has the same fingerprint it prints. The keyring is not in any backup (SECURITY §6.4).

15. **Prove the outside world sees only what it should.** From a machine *not* on the box's network (your laptop at home is fine for the Hetzner target):

```
laptop$ nmap -Pn -p- app.example.com
laptop$ nmap -Pn -6 -p- app.example.com
laptop$ curl -sI https://app.example.com | head -5
laptop$ ssh -o PreferredAuthentications=password -o PubkeyAuthentication=no polyops@ap
```

Expect: exactly three open ports, 22, 80 and 443, in both scans (a full scan can take 10 to 20 minutes). The `curl` shows `HTTP/2 200` (staging) or the maintenance response (prod) over a valid certificate. The last command ends `Permission denied (publickey)`. Any other open port, especially 5432, 2375 or 3000, means a containment failure: close it (`polycarpic-ops lockdown` stops all containers except Caddy), and tell the team-lead before doing anything else.

16. **Make the bootstrap user** (SECURITY C-01, §4.1: an invitation on an empty instance, so the first sign-up cannot be won by a stranger).

```
box$ sudo polycarpic-ops cli user invite you@example.com --bootstrap
```

Expect: it prints "invitation recorded" and emails you a sign-up link; the link works only for that address. If no email arrives within 5 minutes, check the Resend dashboard and the spam folder; `polycarpic-ops logs web | tail -50` shows the send result without any secret.

17. **Open the door, sign up, and set the second step.**

```
box$ sudo polycarpic-ops open prod
```

Expect: `prod is open`. Click your emailed link, create a password, confirm the emailed code, and set up an authenticator app as your second step. Save the backup codes in the password manager. If you are asked to sign in before an invitation exists, step 16 did not complete: run `polycarpic-ops close prod` and repeat from 16.

18. **First backup, on demand.**

```
box$ sudo systemctl start polycarpic-backup.service   # runs backup, then starts the u
box$ sudo systemctl status polycarpic-upload.service
box$ sudo polycarpic-ops status
```

Expect: the service ends `Finished`; `status` shows a backup time (read from `last-upload`) of a few minutes ago, no unaccounted `poly_backup` connections and a heartbeat OK, and your monitor shows the check green. If the upload fails with an access error, the put-only key was pasted wrong; if it fails with a lock error, the bucket's Object Lock is not enabled (step 4).

19. **First restore drill, from your laptop** (this is a v1.0 gate; do it now while it holds nothing, and monthly after).

```
laptop$ infra/drill.sh --bucket polycarpic-<yourname>-backups
```

Expect: it asks for the read key and the age private key file (paste it from the password manager into a temporary file it deletes afterwards), restores into a scratch Postgres on your laptop, and ends `DRILL PASSED` with the checks of §6.7, including `delete attempt: DENIED`, `overwrite: DENIED`, `shorten retention: DENIED` and `bucket retention change: DENIED`. A `FAILED` line names the check. Do not go live with real data until this passes.

7.2 Secondary target: the local-network box

Do §7.1 steps 1 to 4, 12, 14 and 16 to 19 unchanged (names and addresses differ). This section covers what is different: the hardware, the encrypted disk (root and data), the network lockdown and TLS. Do all of it *before* any real data touches the box. One thing to know first: with root encrypted, a box that fails to unlock waits at a console prompt and is unreachable over the network, so have a keyboard and monitor to hand for steps 2, 6 and 7.

1. **Hardware and firmware.** A small x86-64 PC with a working TPM 2.0 (listed as fTPM, PTT or "Security Device" in the firmware; see the pre-purchase checks under Troubleshooting step 6 below), wired Ethernet, an SSD, and a UPS. In the firmware setup (usually Delete or F2 at power-on), do the whole checklist:
   - Update the firmware to the manufacturer's latest.
   - Set a **setup (administrator) password**; store it in the password manager.
   - Secure Boot **on**, in user or deployed mode with the factory keys (not "setup mode").
   - TPM **enabled**.
   - Boot from USB and from network (PXE): **disabled** (the unattended unlock depends on this and on Secure Boot, §3.5). Boot order: the internal SSD only. (Temporarily enable USB for the install in step 2, then disable it again in step 8.)
   - "After power loss": **power on**.

   Expect: the settings stick after a save and reboot. If the machine has no setup-password option, it is not suitable for this role (the firmware password is the control, C-72): choose different hardware.

2. **Install Ubuntu Server 24.04 LTS** from a USB stick (download from ubuntu.com; verify its checksum there). First generate the **recovery passphrase** in the password manager (six or more random words) and write it on the recovery sheet: the installer asks for it now and it becomes the permanent recovery slot. In the installer: language, keyboard, wired network (static address later), "Ubuntu Server (minimized)" is fine; storage: **custom**, with this layout (unmeasured on 24.04, OQ-INFRA-10; if the installer cannot make it, stop and ask):
   - 1 GB EFI system partition (unencrypted, required by the firmware).
   - 1 GB ext4 mounted at `/boot` (unencrypted: kernel and initramfs; the accepted residual, §3.5).
   - The rest of the disk as **one encrypted volume** (LUKS2, passphrase = the recovery passphrase) holding one LVM volume group `polycarpic` with two logical volumes: `root`, 80 GB ext4 at `/`, and `data`, all the remaining space, ext4 at `/srv/polycarpic`.
   - No swap.

   Name the server `polycarpic`; create user `polyops` with a long password you keep in the password manager; tick "Install OpenSSH server" and import your SSH key from GitHub; no extra snaps.

Expect: it reboots to a **passphrase prompt on the console** (type the recovery passphrase; the TPM is not bound yet), then to a login prompt. `lsblk` shows `/` and `/srv/polycarpic` on `crypt` / `lvm` devices and only `/boot` and `/boot/efi` on plain partitions. If it does not boot, re-check that the SSD is first in the boot order.

3. **Give it a fixed address.** In your router, reserve the box's current address for its network card (DHCP reservation). Write the address down.

   Expect: after a reboot of the box, it has the same address. Check from the laptop: `ssh -i ~/.ssh/polycarpic_ed25519 polyops@<address>` works.

4. **Router settings.** Log in to the router. Confirm: no port forwarding rules; UPnP and NAT-PMP **off**; no "DMZ host"; IPv6 either off or its firewall set to block inbound. Screenshot the pages to the password manager's note for this box.

   Expect: all four true. If your router cannot turn UPnP off, replace the router before continuing: it can open the box to the internet without asking.

5. **Provision the host.**

```
laptop$ scp -i ~/.ssh/polycarpic_ed25519 ~/.config/polycarpic/allowed_signers polyops@
box$ sudo polycarpic-ops provision --target home --lan-cidr 192.168.1.0/24
box$ sudo polycarpic-ops provision-check
```

   Expect: `provision-check` ends `ALL CHECKS PASSED`, covering: nftables default-deny with inbound 443 (80 redirects) and SSH from the LAN only; the `DOCKER-USER` drop rule for non-LAN sources; Caddy's publish address equals the LAN address; IPv6 disabled; outbound limited to DNS, NTP and 443 (the check tries port 25 and 8080 and expects failure); sshd key-only without root login; swap off; and, that `/` and `/srv/polycarpic` sit on `dm-crypt` devices (`findmnt`), that `lsblk` shows one `crypt` container under the one partition, with `/` and `/srv/polycarpic` as logical volumes in it and only the EFI partition and `/boot` outside it (SECURITY C-72, S-9), that the LUKS header shows the passphrase slot (the TPM slot is added in the next step, so this check runs again after it), and that the Docker address pool and the fixed subnets of §2.2 do not overlap. The script takes no device argument: it formats nothing.

6. **Bind the encrypted disk to the TPM** (one binding unlocks root and data). The script wraps the method; the candidates are shown so you know what happens.

```
box$ sudo polycarpic-ops disk-bind
   # candidate A (try first): clevis-luks-bind with the tpm2 pin, pcr_ids 7,
   #   + clevis-initramfs, then update-initramfs -u
   # candidate B (if A fails): dracut in place of initramfs-tools,
```

```
   #   + systemd-cryptenroll --tpm2-device=auto --tpm2-pcrs=7
box$ sudo reboot
```

Expect: `disk-bind` asks for the recovery passphrase once, binds the TPM, rebuilds the initramfs and prints the slots (`passphrase` and `tpm2` , nothing else). Then these checks, all of which must pass (together they are "root unlocks after reboot without a prompt"; checks 1 and 3, a plain reboot and a reboot after a kernel update, are the two reboot tests SECURITY S-9 names):

1. **Reboot:** with your hands off the keyboard, the box answers SSH within 3 minutes and `findmnt /` shows a `/dev/mapper/…` device.
2. **Cold start:** shut down, unplug the power for 10 seconds, plug in. Same result.
3. **Kernel update:** install a newer kernel (the next available one; `apt list --upgradable | grep linux-image`) and reboot. Same result. If no newer kernel exists yet, repeat this check after the first one that does, before real data. Until it passes, leave `unattended-upgrades` automatic reboot off on this box (§3.5).
4. `polycarpic-ops provision-check --disk` ends `ALL CHECKS PASSED` : TPM slot and passphrase slot both present on the one container, PCR list is exactly 7.

If any of these waits for a passphrase, the binding did not take: type the recovery passphrase to continue, then see *Troubleshooting step 6* below. PCR 11 is deliberately not used (§3.5). The method that passes is recorded here once OQ-INFRA-10 is measured.

7. **Prove the disk stays locked when the machine is tampered with.** Enter the firmware setup, turn **Secure Boot off**, save, and let it boot, with a keyboard and monitor attached.

   Expect: it stops at the **root unlock prompt on the console** and never reaches the network (so SSH does not answer), instead of unlocking silently. That proves the key is tied to the Secure Boot state, and because root is encrypted too, nothing on the box ran. Type the recovery passphrase only to continue. Then go back into the firmware, turn Secure Boot **on**, save, reboot, and confirm it unlocks by itself again (check 1 of step 6). If it unlocked silently with Secure Boot off, the TPM binding is not what it claims: do not store real data; run `polycarpic-ops disk-bind` again with PCR 7 only and repeat.

8. **Harden the console.** The script sets these and the check verifies them; in plain terms: no automatic console login; the emergency and single-user shells ask for a root password (set with `sudo passwd root` : long, random, saved in the password manager); the bootloader has a password and no editable or recovery entries. Now disable USB boot again in the firmware (the install is finished).

```
box$ sudo polycarpic-ops harden-console
box$ sudo polycarpic-ops provision-check --console
```

Expect: `ALL CHECKS PASSED`. As a hands-on test, power-cycle the box and at the boot menu press `e`: it must ask for the bootloader password.

9. **DNS and TLS: choose one mode and set it before first deploy.** The mode is set by `TLS_MODE` in `/etc/polycarpic/host.env` and the app refuses to start on a mismatch. See 7.3 for each mode, then return here.
10. **Secrets, first deploy, keyring, outside-in check.** Do §7.1 steps 12 and 14 (secrets; first deploy and keyring for prod) with `--target home`; the bootstrap, open, backup and drill steps (16 to 19) follow once the checks in steps 11 and 12 below pass. There is no staging on this box: run the process in staging on the Hetzner server first.

    Expect the same results, except the first-deploy check also prints `served certificate issuer matches TLS_MODE`.

11. **Outside-in scan of your home from outside.** Use your phone on mobile data as a hotspot for the laptop (or a cloud shell). Find your home's public IPv4 address (visit any "what is my IP" page from home) and the box's IPv6 global address if it has one (`ip -6 addr` on the box; disabled means none).

    ```
    laptop-off-LAN$ nmap -Pn -p- <home-public-ipv4>
    laptop-off-LAN$ nmap -Pn -6 -p- <box-ipv6-if-any>
    ```

    Expect: all ports `filtered` or `closed`, none open. Also try browsing to the box's name from mobile data: it must not resolve (real-domain mode) or not be found. Anything open: unplug the box's network cable, then fix the router or firewall (C-76).

12. **Outbound test.**

    ```
    box$ curl -sS --max-time 5 https://github.com -o /dev/null -w "%{http_code}\n"
    box$ curl -sS --max-time 5 http://example.com:8080 || echo blocked
    ```

    Expect: `200` (or a 3xx) for the first; `blocked` (a timeout) for the second.

Troubleshooting step 6. If the TPM slot will not bind: check `systemd-cryptenroll --tpm2-device=list` (or `clevis luks list`) shows a device; confirm the firmware's TPM is enabled and in TPM 2.0 mode; confirm Secure Boot shows "enabled" in `mokutil --sb-state`. If it still fails with both candidate methods, **stop: either this machine will not do (use hardware with a working TPM2 and Secure Boot) or the method does not work on 24.04 (OQ-INFRA-10).** There is no passphrase-at-every-boot fallback (P8 stands), so do not put real data on a box whose disk cannot be bound to the TPM. If the hardware is fine and neither method in step 6 gives an unattended root unlock on Ubuntu 24.04, stop and return to SecEng and the Principal:

the remaining options (a different distribution, a measured unified kernel image, a PIN) change a ruling and are not mine to choose. Do not buy hardware before checking, on the model's own spec sheet or by booting a live USB on it: (1) a TPM 2.0 (fTPM, Intel PTT or discrete) is listed and can be enabled in the firmware; (2) Secure Boot can be turned on and off, with factory keys; (3) the firmware has a setup (administrator) password; (4) USB and network boot can be disabled; (5) it has an "After power loss: power on" option; (6) a live Ubuntu 24.04 USB shows the TPM, `ls /dev/tpmrm0` exists and `systemd-cryptenroll --tpm2-device=list` lists a device. Unmeasured per model; buy from a seller with a return window until steps 6 and 7, including the kernel-update reboot, have passed.

7.3 Home-box TLS, step by step

Mode A (default): real domain, DNS-01 wildcard

1. In Cloudflare create an API token that can **edit DNS records in this one zone only** (My Profile → API Tokens → Create → "Edit zone DNS" template, Zone Resources → Include → Specific zone → your domain). Name it `polycarpic-home-caddy`.
2. `box$ sudo polycarpic-ops set-secret dns_api_token` and paste it. Set `TLS_MODE=real-domain`, `PUBLIC_ORIGIN=https://polycarpic.home.example.com` in `/etc/polycarpic/host.env`.

   Expect: `list-secrets` shows the token, mounted only into `caddy` (`polycarpic-ops show-mounts`).
3. **Make the name resolve on your LAN only.** Preferred: in the router's DNS settings add a local record `polycarpic.home.example.com → <box address>`. If your router cannot, add the line `192.168.1.20 polycarpic.home.example.com` to `/etc/hosts` (macOS/Linux) or `C:\Windows\System32\drivers\etc\hosts` on each device.

   ```
   laptop-on-LAN$ dig +short polycarpic.home.example.com
   laptop-off-LAN$ dig +short polycarpic.home.example.com @1.1.1.1
   ```

   Expect: the first shows the box's address (or use `ping` if you used a hosts file); the second shows nothing. If the second shows an address, you published the box's name: delete that record in Cloudflare.
4. First deploy requests the wildcard. Expect in `polycarpic-ops logs caddy`: `certificate obtained successfully` within about two minutes. Browse to the name from a LAN device: a normal padlock, issuer a public CA. If the order fails with `DNS problem`, the token's zone or permission is wrong; if with `rate limit`, wait an hour and do not retry in a loop.

5. Register your sign-in apps with Google and GitHub using the callback URL `https://polycarpic.home.example.com/api/auth/callback/<provider>`, then `set-secret` each client secret. GitHub asks for `read:user` and `user:email` only.

Mode B (fallback): `.local` with a private CA

1. **Make the CA on your laptop, not the box.**

```
laptop$ infra/ca/ca.sh init --name polycarpic.local
laptop$ infra/ca/ca.sh intermediate --out ./box-intermediate
```

   Expect: `root.crt` (public: this goes on devices), `root.key` (private: it must go into the password manager as a secure note, then be deleted from the laptop) and a folder with the intermediate cert and key. Running `ca.sh show root.crt` must print `Name Constraints: permitted DNS polycarpic.local`. If it does not, the CA is unconstrained: delete it and do not continue.

2. Copy only the intermediate to the box: `scp box-intermediate/* polyops@<address>:/tmp/`, then `box$ sudo polycarpic-ops install-intermediate /tmp/…`. Set `TLS_MODE=private-ca`, `PUBLIC_ORIGIN=https://polycarpic.local`. The root key never leaves the password manager.
3. **Install the root certificate on every device that will sign in:**
   - *macOS:* `sudo security add-trusted-cert -d -r trustRoot -k /Library/Keychains/System.keychain root.crt` (also needed in Firefox: Settings → Privacy & Security → Certificates → Import, tick "trust to identify websites").
   - *iPhone/iPad:* AirDrop `root.crt`, open it, install in Settings → Profile Downloaded; then Settings → General → About → Certificate Trust Settings → enable full trust.
   - *Windows:* double-click, Install Certificate → Local Machine → "Trusted Root Certification Authorities".
   - *Android:* Settings → Security → Encryption & credentials → Install a certificate → CA certificate.
   - *Linux:* copy to `/usr/local/share/ca-certificates/` and run `sudo update-ca-certificates`.
4. **Gate item, once per browser and OS you use (R-20).**

```
laptop$ infra/ca/ca.sh test-constraint
   # issues a certificate for evil.example.com signed under the intermediate and serve
```

Expect: in each installed browser the page shows a certificate error (name constraint violated). If any browser loads it as trusted, that browser ignores the constraint: remove the root from that device and do not use that browser for this box, or move the box to Mode A. Record the result per browser in the ledger via the team-lead.

5. Make `polycarpic.local` resolve: most devices find it by themselves (mDNS). If not, add a router DNS record or a hosts-file line as in Mode A step 3. Open `https://polycarpic.local`: padlock, no warning. OAuth buttons are absent and OAuth routes return 404 by design; sign in with email, password and the second step. Passkeys on a `.local` name are unmeasured: use an authenticator app.

Moving a `.local` box to a real domain

1. Everyone who is an owner or manager must already have an authenticator app or backup codes (passkeys will not survive, because they are tied to the old name; every session ends). Check each member: `box$ polycarpic-ops cli user mfa-methods --all-owners` must show TOTP or backup codes for each. Anyone without: ask them to add one, or use `polycarpic-ops cli user reset-mfa <email>` on their behalf afterwards.
2. Tell the household first: everyone will be signed out once.
3. Do Mode A steps 1 to 4. Change `TLS_MODE` and `PUBLIC_ORIGIN`; `polycarpic-ops redeploy home`.

   Expect: the app starts and the served certificate chains to a public CA. If the app refuses to start with a mode message, the settings disagree: fix `host.env`.
4. Each person signs in with email, password and an authenticator code, then adds Google/GitHub from Settings while signed in. It is never linked automatically by matching email.
5. **Remove the old root from every device** (reverse of the install steps), then delete the intermediate from the box (`polycarpic-ops remove-intermediate`). Keep the root key in the password manager for the record, or destroy it.

7.4 Tertiary target: Vercel + Neon

1. **Neon.** Create a project on Postgres 17 (the value in `infra/postgres-major`), nearest region. Neon applies minor updates itself; they are not ours to pin (§6.1a). In the SQL editor create the login roles `poly_migrator`, `poly_app`, `poly_worker` (the first migration creates the NOLOGIN roles and grants). Copy two connection strings per role: the **pooled** one (host contains `-pooler`) and the **direct** one.

Expect: `select version();` shows the pinned major. Record Neon's restore window for your plan from the plan page (§6.6).

2. **Vercel.** Create the team with MFA on, then two projects from the repository: `polycarpic-web` and `polycarpic-jobs`, build command and output per `infra/vercel/README`. Set environment variables **per project** from the table in that file: `web` gets the public keys and `poly_app` pooled URL; `jobs` gets the keyring, `poly_worker` pooled URL and `CRON_SECRET`; the migrate step gets the `poly_migrator` *direct* URL as a build-only variable on `web` only. Set Production Branch to a branch nobody pushes to, so Vercel never deploys on its own.
3. **Deploy a verified release, by hand.**

```
laptop$ git fetch --tags && git tag -v v1.0.0
laptop$ git checkout v1.0.0
laptop$ infra/vercel/deploy.sh --tag v1.0.0
```

   Expect: `tag signature verified`, two builds, the migration running in the web build and ending `0 pending` on the next run, and two deployment URLs. Then attach your domain to `web`. If the migration step fails, the build fails and the previous deployment keeps serving.

4. **Confirm the split.** `infra/vercel/check.sh` lists variable *names* per project and prints `keyring: jobs only`. A cron route requested without the secret must return 401; the same route on the web project must return 404.
5. **Backups.** Add the weekly export to your calendar and run `polycarpic-ops neon-export` once now (§6.6). Daily imports are the platform ceiling on the Hobby plan; the unlink screen says provider revocation can lag up to a day.

7.5 The operator commands you will use

Run on the box. `polycarpic-ops cli …` runs the application CLI in the one-shot `operator` container (`docker compose --profile operator run --rm operator …`; the link prints to your terminal only, and `docker logs` for that container shows nothing) under its own database login, `poly_operator`, never the worker's; each command writes an `audit_event` (SECURITY C-01, C-35, §4.1).

Command	What it does	You should see
`polycarpic-ops cli user invite <email>`	Invite a person who shares no entity with anyone yet. The only way in besides a membership invitation (there is no open sign-up).	"invitation recorded", an email to them

Command	What it does	You should see
`polycarpic-ops cli user reset-mfa <email>`	Removes that person's second-step methods and ends their sessions; they re-enrol on next sign-in. Use after verifying who is asking, in person or by a call you place yourself.	"sessions revoked: N", an `audit_event`, and an email to the user
`polycarpic-ops cli membership remove-owner <entity> <email>`	The audited backstop for removing a co-owner who cannot or will not act (R-19). Notifies every owner, the target included.	"change.forced recorded", emails sent
`polycarpic-ops cli entity set-founder <entity> <email>`	Sets a new founding owner when the founder has vanished without transferring	As above
`polycarpic-ops cli secrets rotate`	Keyring rotation (§8.12)	"rows remaining under old key: 0"

8. Operations Runbook

Day-2. Same format: command, what you should see, what to do otherwise. After any step that changes the box, run `polycarpic-ops status` and confirm green before you close the laptop.

8.1 Rotate a secret

- **Provider key or token** (Resend, price, BLS, OAuth client secret, Plaid secret, bucket put-key): create the new one at the provider, then `sudo polycarpic-ops set-secret <name>`, then `sudo polycarpic-ops restart web worker`, then revoke the old one at the provider. Expect `status` green and (for the bucket key, which only the `uploader` holds, so restart that one) a successful upload within 5 minutes after `systemctl start polycarpic-upload`. Order matters: revoke last.
- **Database role password:** `sudo polycarpic-ops rotate-db-password app|worker|migrator|backup`. It changes the role in Postgres and the file in one step and restarts the matching container. Expect `healthy`.
- **BetterAuth secret (planned rotation, SECURITY A-33):** `sudo polycarpic-ops auth-secret add` adds a new version that encrypts new data while older versions still decrypt; restart; run `polycarpic-ops cli auth rewrite-secrets` (re-encrypts TOTP seeds, backup codes and OAuth tokens under the newest version); when it reports `0 rows on old versions`, run `polycarpic-ops auth-secret retire <version>`. Expect that nobody is signed out. **Never replace the secret outright** outside an incident: it ends every session and makes every enrolled second step undecryptable (everyone would need `reset-mfa`).
- Annual review: list every secret with `list-secrets` and rotate any that you cannot account for.

8.2 The monthly window (first Sunday, about 45 minutes)

1. Run the restore drill from the laptop (§6.7). Expect `DRILL PASSED`; a failure is SEV2 (RB-5).
2. `box$ sudo apt update && sudo apt full-upgrade`, then `sudo reboot` if it says so. Expect: after about two minutes `polycarpic-ops status` is green. The home box must come back without asking for a passphrase, root and data both (§7.2 step 6 check 1); on the home box this is also where you reboot after a kernel update until the unattended-reboot check has passed (§3.5). Check `ls /var/run/reboot-required` before closing the window: with automatic reboot off, a file older than 7 days raises the pending-reboot alert (§6.5), so a missed window is noticed.
3. Docker: if `apt` held back Docker packages, upgrade them now; containers restart.

4. Check `polycarpic-ops status`: disk under 80 percent, certificate more than 14 days left, GitHub token more than 14 days left.
5. Read the month's alerts and any rolled-back deploys in `/opt/polycarpic/deploy.log`.
6. Check Renovate PRs waiting more than 7 days with the team-lead.
7. **Re-read the operator-account checklist of §7.0 (C-78):** two-step sign-in, own password and recovery address on every account, the registrar transfer lock, and no Hetzner API token left on the laptop (`ls ~/.config/hcloud` is empty). `provision-check` cannot see these.
8. Tell Healthchecks.io the drill happened: `laptop$ curl -fsS -m 10 --retry 3 "$(polycarpic-ops heartbeat-url drill)"` (the helper prints the check's address from the ping key kept in the password manager). Expect: the `polycarpic-drill` check turns green; if it is not pinged for 35 days you get an email.

8.3 Rotate the DNS token (home, real-domain mode)

1. In Cloudflare, create a new token with the same single-zone scope; `box$ sudo polycarpic-ops set-secret dns_api_token`; `sudo polycarpic-ops restart caddy`.
2. Force a renewal test: `sudo polycarpic-ops caddy-renew-test`. Expect: `staging ACME order succeeded` (it uses the issuer's test environment).
3. Delete the old token in Cloudflare. Expect nothing else to change.

8.4 Renew the GitHub read token (every 90 days)

GitHub → Settings → Developer settings → Fine-grained tokens → generate: this repository only; `Contents: read`, `Packages: read` (repository permissions: Metadata read is automatic); 90 days. `box$ sudo polycarpic-ops set-secret github_read_token`, then `sudo systemctl start polycarpic-deploy`. Expect: `deploy.log` shows a poll with "up to date". Then delete the old token and ping the `polycarpic-token` check so its 90-day clock restarts. If the repository and package are public, remove the token (`polycarpic-ops remove-secret github_read_token`) and the reminder.

8.5 Replace the private CA intermediate (60 days before expiry)

1. Laptop: fetch the root key from the password manager to a temporary folder, run `infra/ca/ca.sh intermediate --out ./new-intermediate`, then delete the root key file.
2. Copy and install: `box$ sudo polycarpic-ops install-intermediate /tmp/new-intermediate/…`. Caddy reissues leaves from it. Expect: browsers show no warning, and `polycarpic-ops status` shows the issuer chain. Devices need no change because they trust the root, not the intermediate.

3. If the *root* must be replaced (compromise or 10-year expiry), every device needs the new root installed and the old one removed (§7.3 Mode B step 3), so schedule it; or move to Mode A.

8.6 Roll back a release

```
box$ sudo polycarpic-ops rollback prod
box$ sudo polycarpic-ops status
```

Expect: `rolled back to sha256:… (previous)`, `healthy`, and the bad digest listed under `quarantined` so the poll does not redeploy it. Tell the team-lead; the fix ships as a new release, and quarantine clears when a newer digest appears. If the rollback itself is unhealthy, the schema may have moved in an incompatible way: do not change anything else; use the pre-migration backup (§4.5, §6.8) and ask.

8.7 Update the operator tooling itself

`box$ sudo polycarpic-ops self-update` fetches the newest release's ops bundle, verifies the attestation and tag signature exactly as a deploy does, shows a diff of the deploy script and timer unit against what is installed, and waits for you to type `yes`. Read the diff; if you do not understand a change, answer no and ask. Expect: `self-update complete` and the next poll runs on the new script.

8.8 Add or remove a laptop's SSH key

1. Add: on the new laptop make a key (§7.1 step 2). From a laptop that still works: `ssh polyops@box "sudo polycarpic-ops ssh-key add"` and paste the `.pub` line. Expect: the new laptop can log in. Test before removing anything.
2. Remove (lost laptop): `sudo polycarpic-ops ssh-key list`, then `ssh-key remove <id>`. A lost laptop also means: rotate the bucket read-key, treat its password manager as exposed if it was unlocked, and run SECURITY RB-2 if the laptop held an unlocked session.

8.9 Locked out, or lost a secret

Situation	What to do
Lost every SSH key (Hetzner)	Hetzner console → server → Rescue → enable rescue system, reboot, mount the root disk, add a public key to `polyops`'s `authorized_keys`, disable rescue, reboot. Expect: SSH works with the new key. This is why the Hetzner account and its recovery codes are in the password manager.

Situation	What to do
Locked yourself out with a firewall rule	Hetzner console → Firewalls: edit the rule (the console is unaffected by the firewall).
Home box does not come back after a reboot (it asks for a passphrase at the console)	With root encrypted the box has no network until unlocked, so SSH will not answer: attach a keyboard and monitor and type the recovery passphrase from the recovery sheet. Then find why the TPM no longer matches (firmware update? Secure Boot changed?), and re-bind: `sudo polycarpic-ops disk-rebind`. A passphrase prompt you did not cause (nothing changed) can mean the box was tampered with: run the checks in §8.11 before trusting it.
A household member lost their authenticator	`polycarpic-ops cli user reset-mfa <email>` after verifying them by a channel you control.
Lost the keyring	RB-4: `polycarpic-ops cli secrets init --force`; every connection then shows "relink required"; the ledger is untouched.
Lost the age private key	Existing backups cannot be opened, ever. Generate a new keypair (§7.1 step 3), replace `/etc/polycarpic/age-recipients.txt`, run a manual backup, and consider the old backups gone. This is why the key has two homes.
Lost the recovery passphrase of the home box's disk	While the box still unlocks by itself: add a new passphrase slot (`sudo cryptsetup luksAddKey`) and remove the old one. If the TPM no longer unlocks it and the passphrase is gone, restore from backup onto new disk (§8.10).

8.10 Restore from backup and replace a box

1. **Get the keys in front of you:** age private key, the keyring offline copy, the provider secrets from their consoles, the recovery sheet.
2. **Build the replacement** by repeating §7.1 steps 6 to 11 (Hetzner) or §7.2 (home) *with a new, empty data volume*, but do not yet point DNS at it and do not run `first-deploy`.
3. **Restore the data** on your laptop first (it needs the age private key, which must not touch the new box): `laptop$ infra/restore.sh --bucket … --to-file ./restore.tar.age-decrypted` decrypts the latest base backup and WAL into a directory; copy that directory

(plain data) to the new box over SSH into `/srv/polycarpic/restore/`, then `box$ sudo polycarpic-ops restore-apply prod`. Expect: Postgres recovers to the latest WAL and reports `recovery complete`, then the verifier prints `trial balance: ok; hash chain: ok`. Delete the plain copy from the laptop when done (`rm -rf`) and from `/srv/polycarpic/restore/`.

   The plain data exists on your laptop and then on the new box's encrypted volume only. If the laptop is shared or unencrypted, use a throwaway server for the decrypt step instead.

4. **Re-enter the secrets** (§7.1 step 12), restore the keyring file from the offline copy (`polycarpic-ops restore-keyring`; `keyring-fingerprint` must match what you wrote down), or generate a new keyring and relink every connection (RB-4).
5. **Deploy the last release** (`polycarpic-ops first-deploy prod`), open it, switch DNS to the new address, and confirm in the outside-in scan (§7.1 step 15).
6. Run a backup and the monitor check; run the restore drill once more within the week.
7. Time yourself. RTO is 4 hours on Hetzner; if you took longer, write down the slow step and tell the team-lead so §7 improves.

Temporary landing on Hetzner for a lost home box: do the same on a Hetzner server with `--target hetzner`, using a temporary DNS name. Household members use that name until the home box is rebuilt. Real-domain mode on the home box is unaffected by this.

Neon: create a new Neon project, restore with `polycarpic-ops neon-restore` from the latest weekly export (decrypt on the laptop), update the Vercel variables, redeploy with `infra/vercel/deploy.sh`.

8.11 Suspected compromise (commands for SECURITY RB-1, RB-2)

1. **Stop the bleeding.** `box$ sudo polycarpic-ops lockdown`: Caddy serves the maintenance page and every other container stops. Expect: the outside scan shows only Caddy. If you cannot log in, disable the server's network in the provider console (Hetzner) or unplug the cable (home).
2. **Keep the evidence.** Hetzner: take a server snapshot in the console. Home box: do not power off if you can avoid it; copy `deploy.log` and `docker logs` output to the laptop.
3. **If the worker or keyring may be exposed (RB-1):** revoke every bank token at its provider (Plaid `/item/remove`, SimpleFIN, broker OAuth), then after the replacement is up, generate a new keyring and relink. If only the keyring leaked, `polycarpic-ops cli secrets rotate` (§8.12) is enough; if the database and the keyring may both have been exposed, rotation is not enough.

4. **Rotate every operator secret** (§8.1), the BetterAuth secret by replacement this time (sessions all end), the bucket keys and the DNS token. Remove the SSH keys you cannot account for (§8.8).
5. **Rebuild** from a clean attested image on a fresh host (§8.10), restoring the last backup taken before the compromise, and verify it against the hash chain and against the head of the last export an owner holds off the box (RB-2 step 6).
6. Postmortem within 5 business days (SECURITY §10).

8.12 Rotate the credential keyring

`box$ sudo polycarpic-ops cli secrets rotate`. Expect: it adds key *n+1*, re-seals every connection under it, and ends `rows remaining under old key: 0; removed key n; audit_event written`. Save the new keyring offline immediately: `polycarpic-ops keyring-show` prints it once; replace the password-manager note and the printed sheet, and confirm with `keyring-fingerprint`. If it stops midway it is safe to run again.

8.13 Decommission

1. Export everything each household member wants to keep (app export carries the hash-chain head); run a final backup; wait for it to appear in the bucket.
2. Revoke every bank connection from the app (Settings → Connections → Unlink) and confirm each at its provider.
3. Hetzner: delete the server, the volume, the firewall, the SSH key and the project. Home box: remove the intermediate/DNS token, then securely erase the disk (`sudo cryptsetup erase /dev/…` then `blkdiscard`, or physically destroy it).
4. Delete the DNS records and the DNS token; remove the private CA root from every device; delete the Resend domain, OAuth apps, monitors and the GitHub token.
5. Backups: nothing to delete; they expire in 35 days and cannot be removed earlier. Destroy the age private key only after you are certain you will never restore (or after the last backup expires).

9. Rulings & Asks on Other Owners

Decisions on the devops items ARCH §7 and SECURITY §12.3 left open. Each cites its measurement or is tagged (unmeasured); an unmeasured claim is a first-deploy check, not a promise.

9.1 Rulings (D-1 to D-16)

Built by: D-2, D-6, D-8 in POLY-95; D-5 in POLY-100; D-7 in POLY-97; D-10 in POLY-103; D-13 to D-15 in POLY-99; D-11, D-12 in POLY-121; D-16 in POLY-120.

#	Item (source)	Ruling
D-1	Secrets as files, scoped per container (SECURITY §6.2, C-74)	Confirmed as ARCH §7.5 states it. File inventory and per-container mounts: §2.2 and §3.2. Non-secret host config (`TLS_MODE` , `PUBLIC_ORIGIN` , instance name) lives in `/etc/polycarpic/host.env` . On the home box the secrets directory sits on the TPM-unlocked partition, so a pulled disk yields no secrets; on Hetzner it sits on the root disk and the data volume's LUKS key is a file on that disk, which protects a detached volume only, not a captured server (said plainly in §3.2).
D-2	Only Caddy publishes ports; firewall outside the host (C-70, C-71)	Confirmed. Hetzner Cloud Firewall admits 22, 80, 443 on IPv4 and IPv6. SSH stays open to the internet with key-only login, because the operator's address changes (a restriction by address is optional and §8.9 says how to recover). Acceptance is behavioural: the compose-policy lint in CI and the outside-in scan after every provisioning (§7.1 step 15), not the config alone.
D-3	Backup store and mechanism (§6.6, C-73)	**Backblaze B2 bucket (EU) with Object Lock in compliance mode, 35-day retention, put-only box key, separate read key on the operator's laptop**, written over HTTPS 443 through B2's S3-compatible endpoint, `rclone` as the uploader, `age` to a recipient public key. Why not the Storage Box default: the C-76 outbound allowlist (DNS, NTP, 443) blocks SFTP on 22/23 on the home box, and a Storage Box "append-only" mode is *(unmeasured)*. A Storage Box stays a documented fallback on Hetzner once the cannot-delete drill passes against it. One mechanism on all targets is simpler to drill and to explain. Hetzner Object

#	Item (source)	Ruling
		Storage is a noted alternative pending verification of its lock; the Storage Box is fallback only. Nightly set is base backup + dump + globals, WAL continuous (§6.1).
D-4	RPO and RTO (SECURITY §6.6 "DevOps's call"; R-5)	RPO 15 minutes, RTO 4 hours on Hetzner; home box RTO 1 day with a Hetzner break-glass at 4 hours, no spare hardware; Vercel/Neon per §6.4. Retention 35 days, enforced by the store; this bounds R-5 at 35 days.
D-5	Pull-based deploy (C-83)	Confirmed and made concrete in §4.4. Added: the ops bundle is extracted from the attested image; the prod tag's signer list (`allowed_signers`) is installed at bootstrap from the operator's own machine, never fetched from the repository; deploy failure rolls back automatically within 120 seconds; the poll quarantines a failed digest.
D-6	Connection-pooler mode on both targets (§12.3)	Compose targets: **no pooler in v1**; the app's own pool holds connections and `set_config(…, true)` is transaction-local, so reuse is safe. A pooler service in compose is refused by the policy lint unless it declares transaction mode. Neon: the runtime uses the **pooled** endpoint (PgBouncer transaction mode); migrations use the **direct** endpoint. CI: on a fixed pooler-reuse test, connect through each target's endpoint (a local PgBouncer in transaction mode stands in for Neon, and a real Neon branch is used on the scheduled run), set the GUC inside a transaction, commit, and assert a later transaction on the same backend sees it empty.
D-7	Provisioning tool	Committed `hcloud` CLI scripts and a cloud-init file (`infra/hetzner/`), idempotent, plus the on-box `polycarpic-ops provision`; not OpenTofu. Reason: a state file is one more thing for a novice to lose, and one server, one firewall and one volume do not need a planner. Revisit with a second server. The scripts are the source of truth: a console change that the scripts do not make gets reverted on the next run.

#	Item (source)	Ruling
D-8	Postgres logging (C-75)	A mounted configuration fragment sets the three C-75 values; the `image` job's compose-run Postgres runs `SHOW` on each and fails if any differs. Same assertion runs against the staging and local-box containers in `provision-check`.
D-9	Local-box provisioning checks (C-76) and staging isolation (C-77)	`provision-check` is the check script both controls ask for: nftables rules, the `DOCKER-USER` rule, publish binding, sshd, IPv6, outbound allowlist, swap, TPM slots, console hardening, and that the staging and prod secrets directories share no file and staging's `PLAID_ENV` is `sandbox`. Outside-in scan: §7.2 step 11.
D-10	Monitoring and alerts (ARCH §7.4, C-76 heartbeat)	Two independent paths: app email through Resend, and an outside dead-man's-switch monitor (Healthchecks.io, free tier) that does not depend on the box or Resend (§6.5). Public health endpoint says only ok or not.
D-11	Vercel deploy path	The operator deploys a signature-verified tag by running `infra/vercel/deploy.sh` from the laptop; Vercel's own Git deployment is pointed at a branch nobody pushes to. No Vercel deployment credential lives in GitHub, consistent with the C-83 principle that nothing GitHub holds can move a deployment.
D-12	OQ-26: backups on Vercel	Neon restore window plus a weekly operator-run encrypted export; no nightly export, no GitHub-held credential; revisited with SecEng only if real data lands on the target (OQ-INFRA-1). §6.6.

#	Item (source)	Ruling
D-13	Backup and upload split (SECURITY C-79, A-36; ARCH §7.2)	Two services. `backup`: the only holder of `db_password_backup`, on the internal `backup-net` with Postgres, no internet, no bucket client. `uploader`: the only holder of the put-only key, on `upload-net`, no database credential. The spool between them holds only `.age` files; encryption happens in the pipe and in `archive_command`, so the Postgres image gains `age`. Subnets fixed in compose and in the first `pg_hba.conf` lines. Slot and WAL caps, `--no-role-passwords`, the slot alert and the connection-accounting check as in §2.2, §6.1, §6.5. Base backup uses `-X none` (tar to stdout cannot stream WAL), *(unmeasured)*, OQ-INFRA-11.
D-14	Bad-migration restore (SECURITY §12.3 ask; ARCH §7.2 "30 minutes")	Runs off the box: the encrypted pre-migration file is fetched to the laptop, decrypted with the age key, sent back as plain data to the encrypted volume, restored into a scratch database for comparison. RTO becomes **60 minutes**. No unencrypted copy on the spool (§6.4). Acceptance of 60 minutes is Q8.
D-15	Backup age and last-backup time (ARCH §7.4)	A host state file (`/var/lib/polycarpic/state/last-upload`) written by the `uploader` after a verified upload; read by `polycarpic-ops status`, the 26-hour alert and the backup heartbeat; not served by the app (§6.5).

#	Item (source)	Ruling
D-16	Home-box root encryption (SECURITY C-72, T-85)	One LUKS2 container holding root and data as logical volumes, unlocked by one PCR-7 TPM binding with a recovery passphrase; EFI and `/boot` unencrypted (accepted residual). Method proven at the first home-box deploy (OQ-INFRA-10; candidates Clevis, then dracut); none works: stop and return to SecEng and the Principal. Automatic reboot stays off until the kernel-update reboot check passes.

9.3 Asks on other owners

- **architect:** Pin the Postgres major and record it in a file CI, compose and the Neon runbook all read. Specify the loopback admin listener (TB-13) and the `check-config` and `migrate --plan` subcommands that §4.5 relies on.
- **seceng:** Review the `provision-check` script (POLY-97) as the evidence for C-70, C-71, C-72, C-76 and C-77.
- **backend-lead:** the `*_FILE` config loader, `polycarpic cli` subcommands named in §7.5 and §8.1 (`user mfa-methods`, `auth rewrite-secrets`, `secrets init --force`, `membership remove-owner`, `secrets rotate`), the log schema agreed before POLY-M1 logs anything, and the commit label on the image.
- **qa-engineer:** the pooler-reuse test, the compose-policy lint fixtures, the topology-check fixtures, the drill script's assertions (§6.7, now with the overwrite, shorten and bucket-retention refusals and the FORCE-RLS row count), the C-79 compose-lint fixtures (internal network, subnet equals `pg_hba`, mounts), the route-policy test (A-34), and Playwright smoke against staging after each deploy.
- **team-lead:** add the INFRA row to the Artifacts table; run the monthly drill reminder issue until it is automated.

10. Open Questions

OQ-INFRA rows are measurements or rulings with an owner and a trigger.

10.2 Measurements and rulings

#	Question	Owner	Resolve by (issue)
OQ-INFRA-1	Nightly Neon export credential in GitHub (TB-12). Default: no; the weekly manual export stands.	secceng + Principal	Trigger: real data first lands on the Vercel target
OQ-INFRA-2	Does Ubuntu 24.04 ship signed PCR policies so PCR 11 can be added? Can the initramfs be measured (a signed unified kernel image, or PCR 9 re-sealed before each update's reboot) without locking the box?	devops-engineer	First home-box deploy (**POLY-120**)
OQ-INFRA-3	Browser enforcement of name constraints on a locally installed root, per browser and OS (R-20)	devops-engineer	First private-CA deploy (**POLY-120**)
OQ-INFRA-4	B2: compliance-mode Object Lock on a bucket-default retention, the exact capability set of the put-only key (does rclone need `listBuckets`?), the lifecycle rule, and the cannot-delete, cannot-overwrite, cannot-shorten and cannot-change-bucket-retention tests with the put-only key (C-73), and the storage cap and alert	devops-engineer	Before the first backup (first drill) (**POLY-105**)
OQ-INFRA-5	Container memory caps, plan size, volume size, disk and egress use	devops-engineer	After POLY-M1 lands, then at GA (**POLY-117**)
OQ-	Neon restore window on the plan in use; Vercel Hobby limits at deploy time	devops-	When the tertiary target

#	Question	Owner	Resolve by (issue)
INFRA-6		engineer	is first stood up (**POLY-121**)
OQ-INFRA-7	Hetzner current plan name and price; Hetzner Cloud Firewall rule counts and IPv6 behaviour	devops-engineer	At the first order (**POLY-104**)
OQ-INFRA-8	Every command in §7 and §8 verified once on a real machine; the runbook is corrected wherever it differs	devops-engineer	First staging deploy (POLY-M1) (**POLY-104**)
OQ-INFRA-9	Healthchecks.io free-tier limits (check count, minimum period and grace, ping-key behaviour, `/fail` handling)	devops-engineer	At account creation (**POLY-104**)
OQ-INFRA-10	Prove an unattended TPM unlock of an encrypted **root** on Ubuntu 24.04 server: candidate A (installer LVM+LUKS2, Clevis tpm2 pin PCR 7, clevis-initramfs), then B (dracut, systemd-cryptenroll). Pass criteria: §7.2 step 6 checks 1 to 4 and step 7. If neither works, stop and return to SecEng and the Principal.	devops-engineer	First home-box deploy, before any real data (**POLY-120**)
OQ-INFRA-11	The backup set as designed: `pg_basebackup –Ft –X none –D –` through `age` produces a base that recovers with the archived WAL; `archive_command` with `age` keeps up and fails safe; `poly_backup` with `BYPASSRLS` and a read-only default dumps every row; no slot remains after a run.	devops-engineer	First staging deploy that has a backup (before the first drill) (**POLY-105**)

Generated by the `devops-engineer` agent. Update when ARCH or SECURITY decisions change; log decisions in process/DECISIONS.md.